



湖南大学
HUNAN UNIVERSITY

第五章 树

湖南大学计算机学院

小班讨论



分成六组:

第1-2组, 《算法导论》第3章思考题3-1、第16章思考题23-3

第3-4组, 《算法导论》第6章思考题6-2、第8章思考题8-3

第5-6组, 《算法导论》第7章思考题7-1、第11章思考题11-1

注意: 《算法导论》第四版中的第16章是《算法导论》第三版中的第17章

概述



- 机场跑道预定系统:
 - 定义
 - 如何利用列表解决一系列飞机起飞时间节点的安排问题。
 - 二叉搜索树
 - 操作
- 阅读材料：《算法导论》第10章，第12.1-12.3节。



<http://izismile.com/tags/Gibraltar/>

跑道预定系统



- 问题定义:

单一跑道 (繁忙)

预定起飞时间

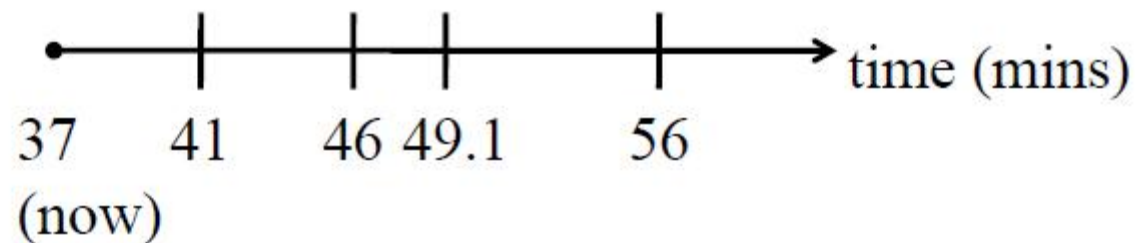
- 维护一个集合的起飞时间节点
- 给定一个新的起飞时间节点申请 t
- 把 t 加入到集合中: 新的时间节点 t 与其他所有时间节点的间隔小于3分钟
- 当飞机起飞, 则把它的时间节点 t 从集合中删除



跑道预定系统



例子:



– $R = (41, 46, 49.1, 56)$

– requests for time: 新时间节点请求

• $44 \Rightarrow \text{reject}$ (46 in R) 拒绝

• $53 \Rightarrow \text{ok}$ 允许

• $20 \Rightarrow \text{not allowed}$ (already past) 不允许, 超过边界

对于有效实现的思路?

跑道预定系统问题分析



- 需要有序：飞机要按序起飞
 - 注意：是完整的顺序，不是堆那种只保证最小或最大
- 频繁插入和删除：经常会有飞机起飞申请
 - 注意：数组不合适，插入与删除成本大
- 查找频繁：飞机起飞申请要判断合法性
 - 注意：排序的链表不合适



一些选项



- 保持R作为一个未排序的列表
 - 缺点：需要线性时间来搜索冲突
 - 优点：可以在 $O(1)$ 时间内插入 t
- 保持R为一个排序数组（每次插入后重新排序）
 - 缺点：需要很多时间来插入元素
 - 优点：3分钟的检查可以在 $O(\log n)$ 时间内完成——使用二分搜索，找到最小的 i 使得 $R[i] \geq t$ （下一个较大的元素）
- 比较 t 与 $R[i]$ 和 $R[i-1]$

数组插入效率低，需要更快的插入算法及数据结构



二叉搜索树



二叉搜索树的每个结点都包含一个关键字key，同时包含属性left、right和parent，它们分别指向结点的左孩子、右孩子和双亲。如果某个孩子结点的父结点不存在，则相应属性值为NIL。根结点是树中唯一父指针为NIL的结点。

对于树中的任意一个节点，其左子树中的所有节点的关键字值，都必须小于该节点的关键字值；类似地，其右子树中的所有节点的关键字值，都必须大于该节点的关键字值。

二叉搜索树与链表的关系



链表

每个点维护一个前驱 (pre) 和后继 (next)

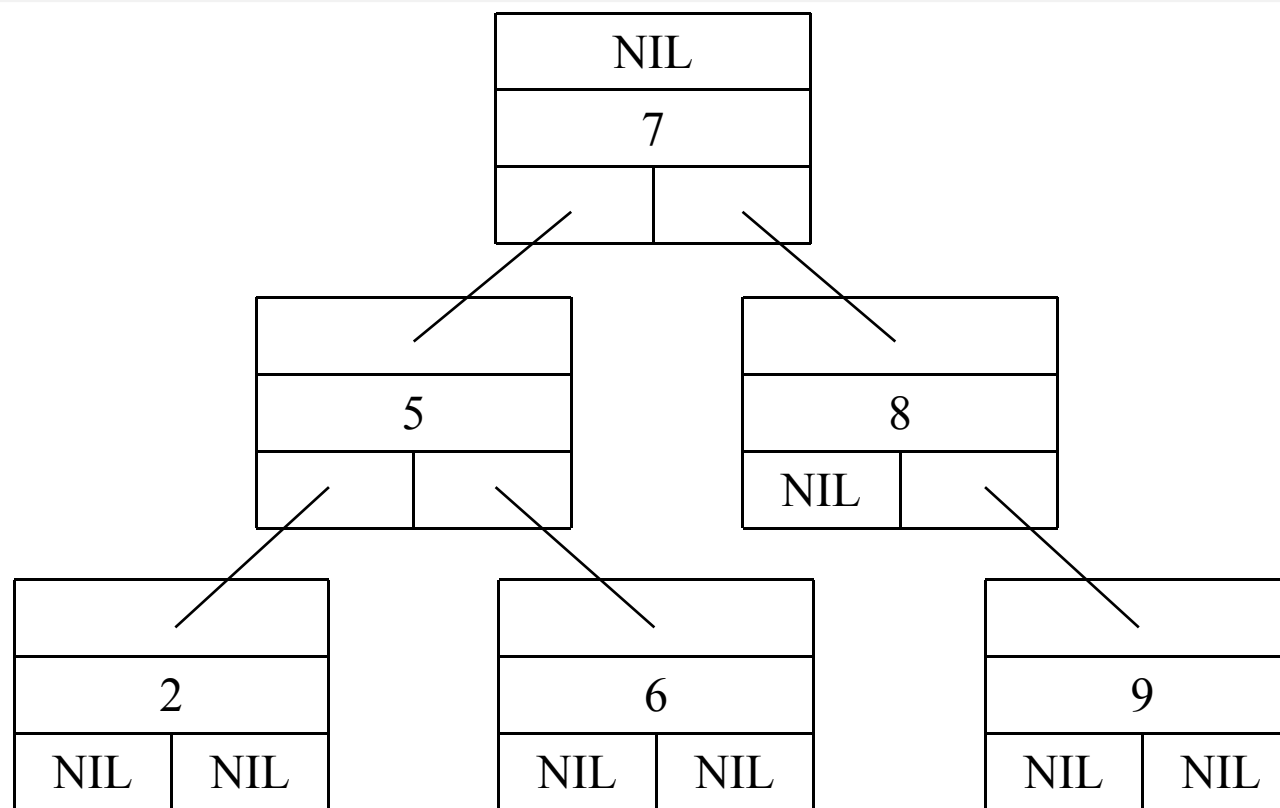
pre
key
next

二叉搜索树

每个点维护一个父亲 (parent) 和两个后继，两个后继分别命名为左 (left) 和右 (right)

parent	
key	
left	right

二叉搜索树示例



包含6个结点、高度为2的二叉搜索树

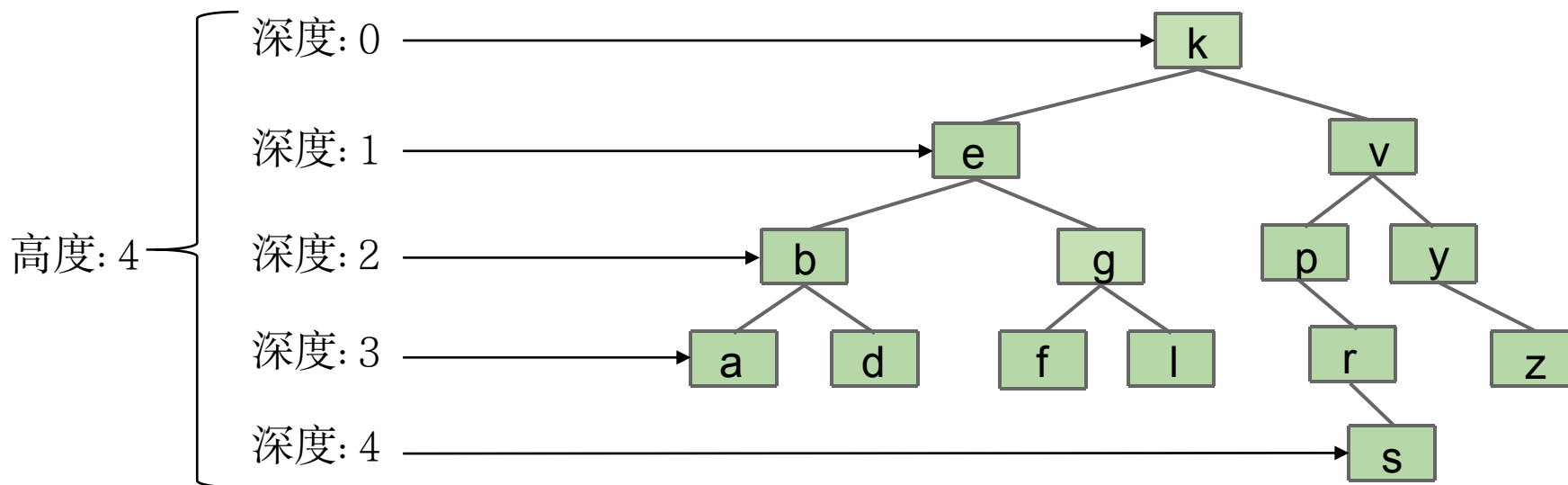
二叉搜索树高度与深度



高度和平均深度是二叉搜索树 (BST) 的重要属性。

- 节点的深度: 该节点到根节点的距离, 例如: $\text{depth}(g) = 2$
- 树的高度: 树中最深叶节点的深度, 例如: $\text{height}(T) = 4$
- 树的平均深度: 所有节点深度的平均值,

例如: $(0 \times 1 + 1 \times 2 + 2 \times 4 + 3 \times 6 + 4 \times 1) / (1 + 2 + 4 + 6 + 1) = 2.35$



查询二叉搜索树

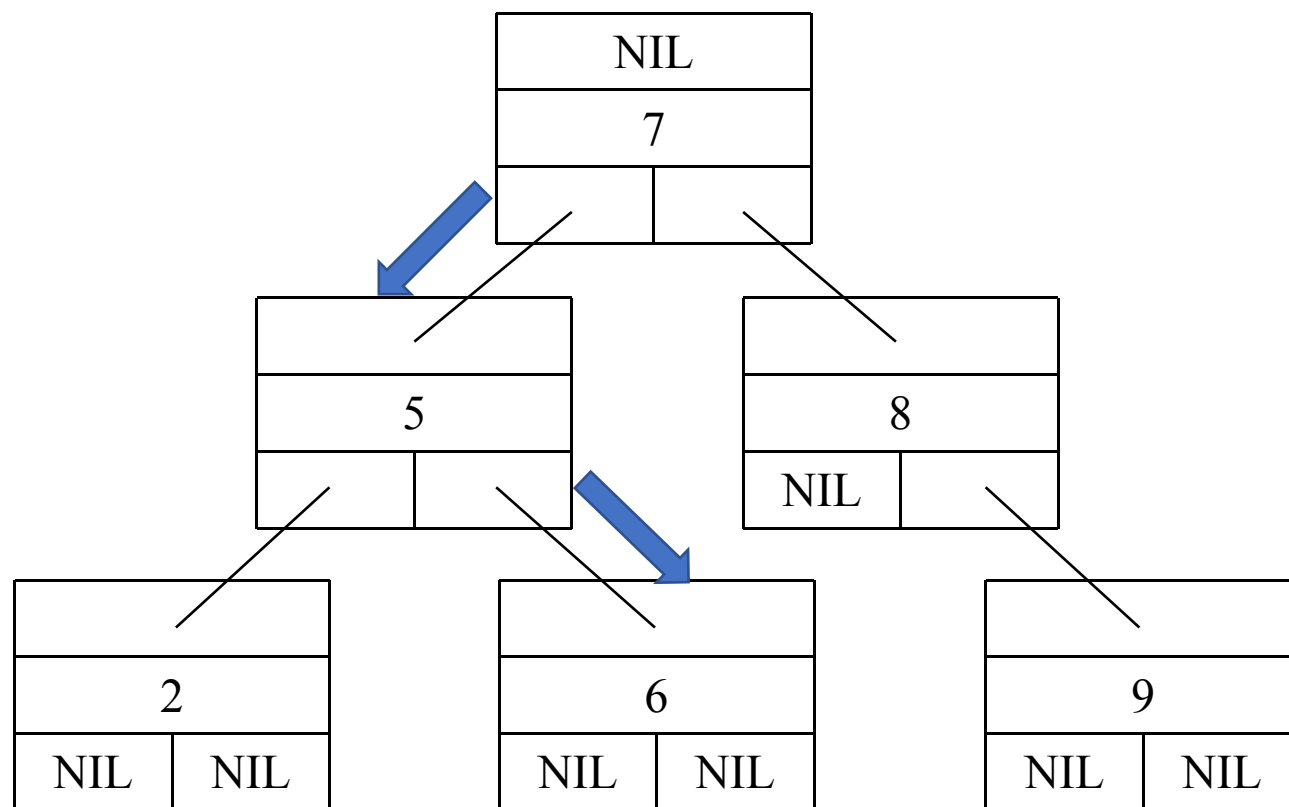


在一颗二叉搜索树中查找一个具有给定关键字的结点，输入一个指向树根的指针和一个关键字 k ，如果这个结点存在，`TREE-SEARCH`返回一个指向关键字为 k 的结点的指针，否则返回`NIL`。

`TREE-SEARCH(x,k)`

- 1 If $x == \text{NIL}$ or $k == x.\text{key}$
- 2 return x
- 3 If $k < x.\text{key}$
- 4 return `TREE-SEARCH(x.left,k)`
- 5 else return `TREE-SEARCH(x.right,k)`

查询二叉搜索树



查询关键字为6的结点，
从树根开始， $6 < 7$ ，故沿着
left指针查找左子树； $6 > 5$ ，
故沿着right指针查找右子树；
最终找到6

迭代查询二叉搜索树



ITERATIVETREE-SEARCH(x, k)

```
1 while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$ 
2     If  $k < x.\text{key}$ 
3          $x = x.\text{left}$ 
4     else
5          $x = x.\text{right}$ 
6 return  $x$ 
```

我们可以采用while循环来展开递归，用一种迭代方式重写这个过程，对于大多数计算机，迭代版本的效率要高得多。

最大关键字元素

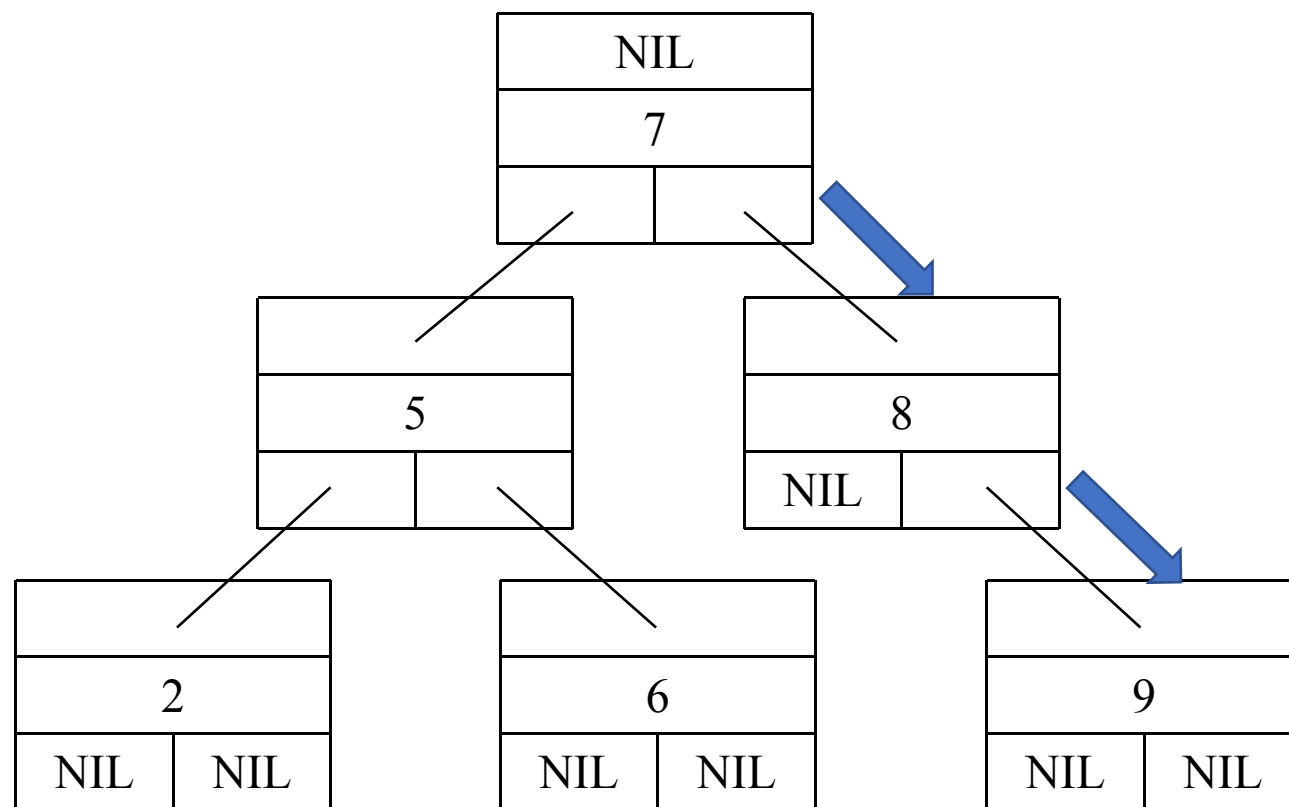


TREE-MAXIMUM(x)

```
1 while x.right ≠ NIL
2   x = x.right
3 return x
```

- 如果**结点没有右子树**，那么由于x左子树中的每个关键字都至少小于或等于x.key，则以x为根的子树中的**最大关键字是x.key**
- 如果**结点有右子树**，那么由于其左子树中没有关键字大于x.key，且在右子树中的每个关键字不小于x.key，则以x为根的子树中的最大关键字一定在**以x.right为根的子树中最大关键字**

最大关键字元素



最小关键字元素



TREE-MINIMUM(x)

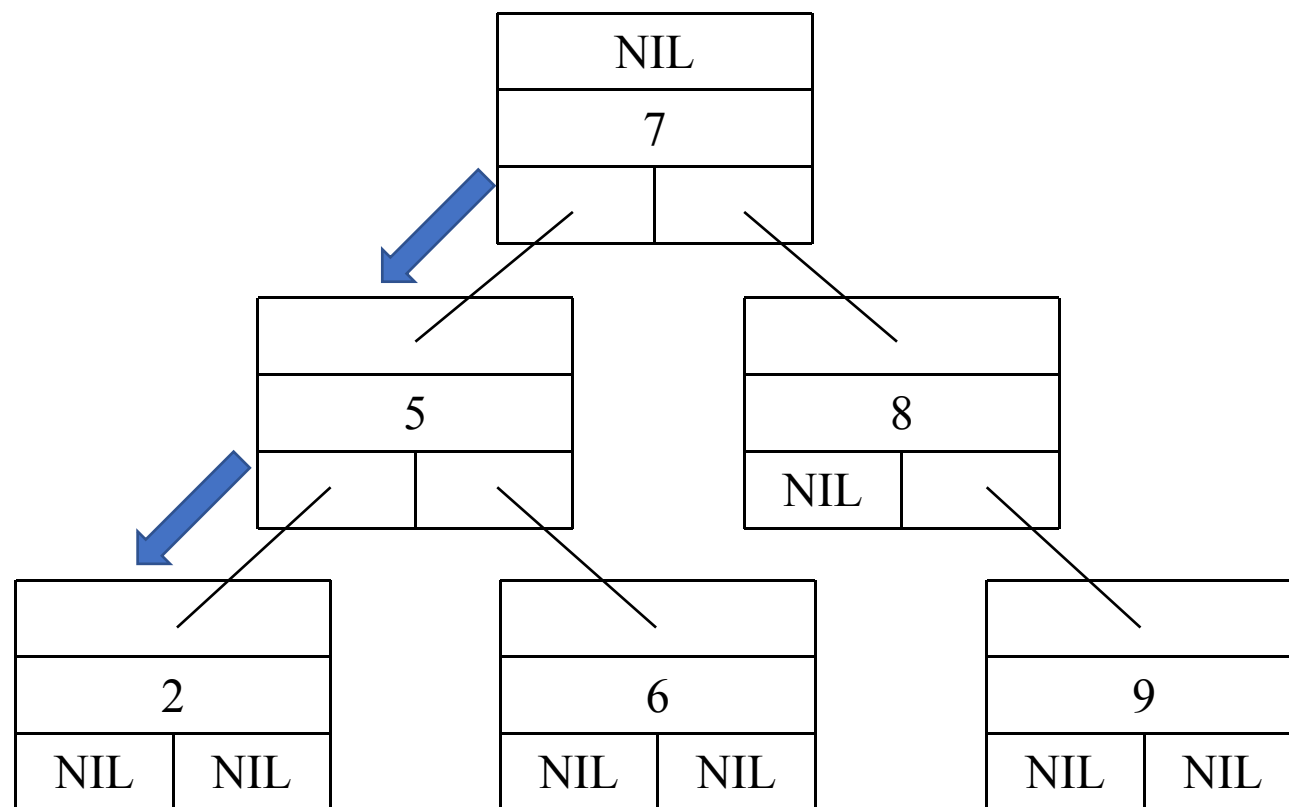
1 while $x.\text{left} \neq \text{NIL}$

2 $x = x.\text{left}$

3 return x

- 如果**结点没有左子树**，那么由于x右子树中的每个关键字都至少大于或等于x.key，则以x为根的子树中的**最小关键字是x.key**
- 如果**结点有左子树**，那么由于其右子树中没有关键字小于x.key，且在左子树中的每个关键字不大于x.key，则以x为根的子树中的最小关键字一定在**以x.left为根的子树中最小关键字**

最小关键字元素



查询二叉搜索树



上面三个过程在一颗高度为 h 的树上均能在 $O(h)$ 的时间内执行完，它们所遇到的结点均形成了一条从树根向下的简单路径

二叉搜索树高度与深度



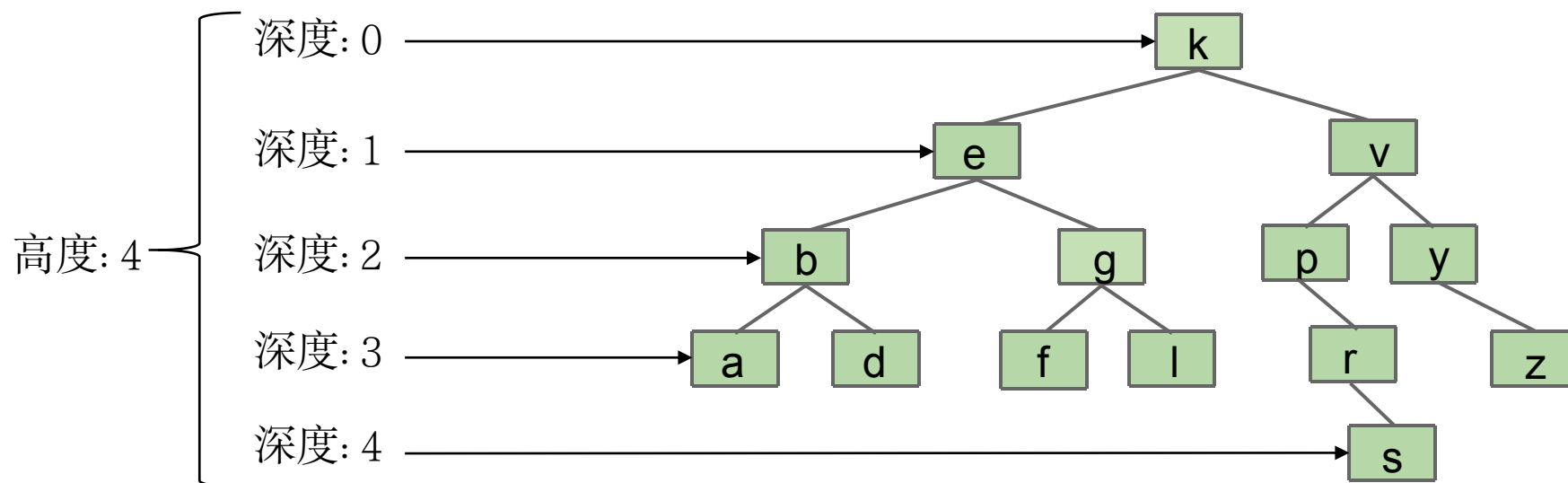
树的高度和平均深度决定了二叉搜索树操作的运行时间。

- 树的高度 → 决定查找节点的最坏情况运行时间

示例：最坏情况为查找 s，需要 5 次比较（高度 + 1）

- 平均深度 → 决定查找节点的平均情况运行时间

示例：平均情况需要 3.35 次比较（平均深度 + 1）



后继和前驱



TREE-SUCCESSOR(x)

```
1 if  $x.right \neq NIL$ 
2   return TREE-MINIMUM( $x.right$ )
3  $y = x.p$ 
4 while  $y \neq NIL$  and  $x == y.right$ 
5    $x = y$ 
6    $y = y.p$ 
7 return  $y$ 
```

- 二叉搜索树的应用需求包括找到比某个元素 x 大的元素中最小的（后继）和比某个元素 x 小的元素中最大的（前驱）
- 以找后继为例，如果 x 有右孩子，那直接调用**TREE-MINIMUM($x.right$)**
- 如果 x 没有右孩子，那看 x 是否是其父亲的左孩子；如果不是其父亲的左孩子，那看 x 父亲是不是 x 爷爷的左孩子；如此迭代，直到找到 x 某个祖先是其父亲的左孩子

二叉搜索树的插入



TREE-SEARCH(T,z)

```
1 y == NIL
2 x == T.root
3 While x ≠ NIL
4     y = x
5     If z.key < x.key
6         x = x.left
7     else x = x.right
8 z.p = y
9 If y == NIL
10    T.root = z    //tree T was empty
11 elseif z.key < y.key
12     y.left = z
13 else y.right = z
```

要将一个新值 v 插入到一颗二叉搜索树 T 中，需要调用过程TREE-INSERT。该过程以结点 z 作为输入，其中 $z.key = v$, $z.left = NIL$, $z.right = NIL$ 。这个过程要修改 T 和 z 的某些属性，来把 z 插入到树中的相应位置上。

构造一颗二叉搜索树



插入7

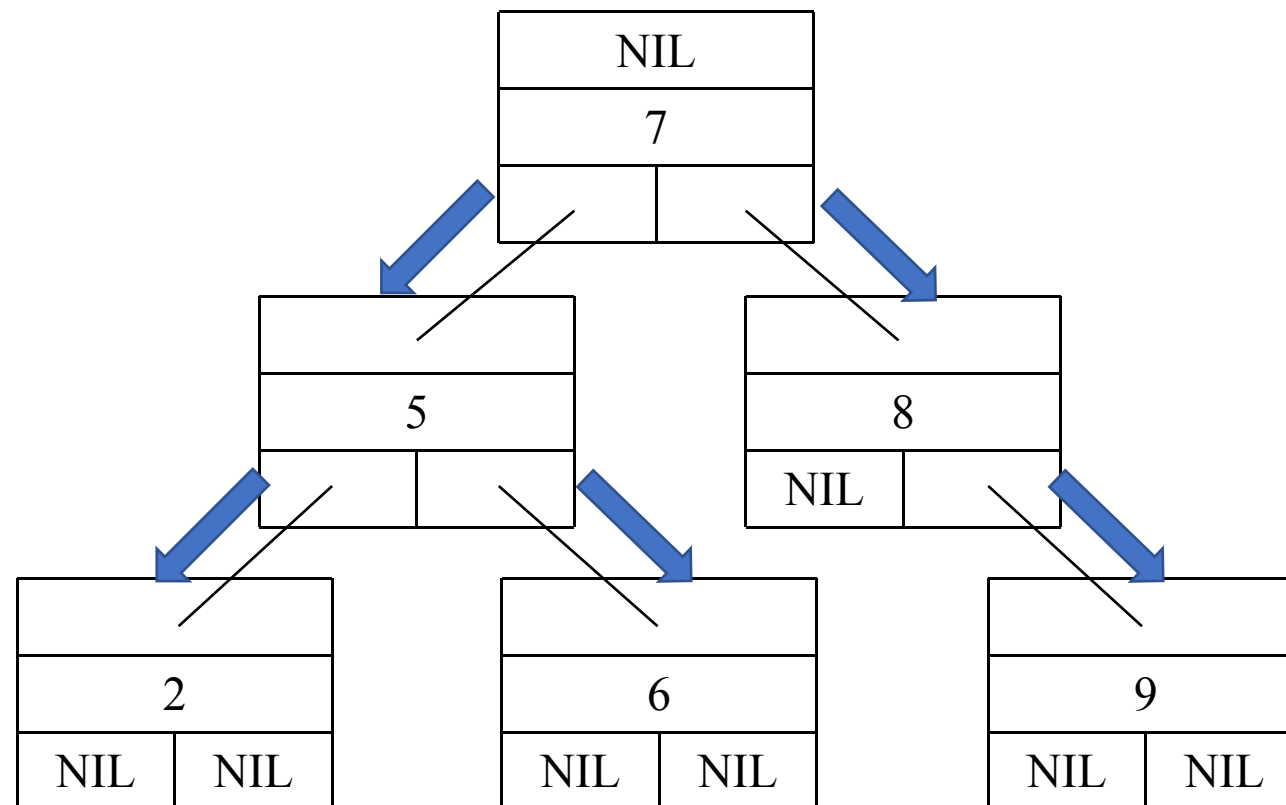
插入5

插入6

插入8

插入2

插入9



从一棵二叉搜索树T中删除一个结点z



从一棵二叉搜索树T中删除一个结点z的整个策略分为三种基本情况(如下所述)，但只有第三种情况有点辣手。

- 如果z没有孩子结点，那么只是简单地将它删除，并修改它的父结点，用 NIL 作为孩子来替换z。
- 如果z只有一个孩子，那么将这个孩子提升到树中z的位置上，并修改z的父结点，用z的孩子來替换z。

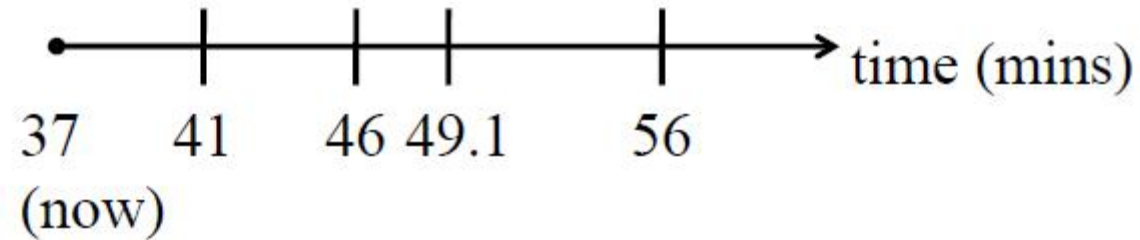
从一棵二叉搜索树T中删除一个结点z



从一棵二叉搜索树T中删除一个结点z

- 如果z有两个孩子，那么找z的后继y(一定在z的右子树中)，并让y占据树中z的位置。z的原来右子树部分成为y的新的右子树，并且z的左子树成为y的新的左子树。

BST 作为数据结构



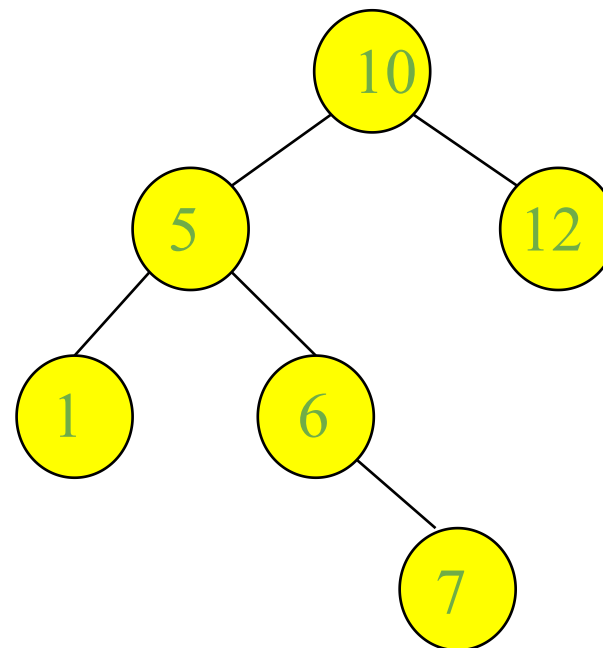
数据结构的操作

- 插入：插入给定键值 k 的节点
- 搜索：搜索包含键值 k 的节点（如果存在）
- 下一个最大节点：找出当前节点 x 的下一个最大节点
- 找最小节点：找出当前节点 x 为根节点的子树的最小节点
- 删除节点 x

搜索



- 搜索：搜索包含键值k的节点（如果存在）
递归查找左节点或右节点，
- 直到找到k或者不在列表中为止



Search(7)
Search(8)

下一个最大节点



next-larger(x): 下一个最大节点: 找出当前节点x的下一个最大节点

- If $\text{right}[x] \neq \text{NIL}$ then
return minimum($\text{right}[x]$)

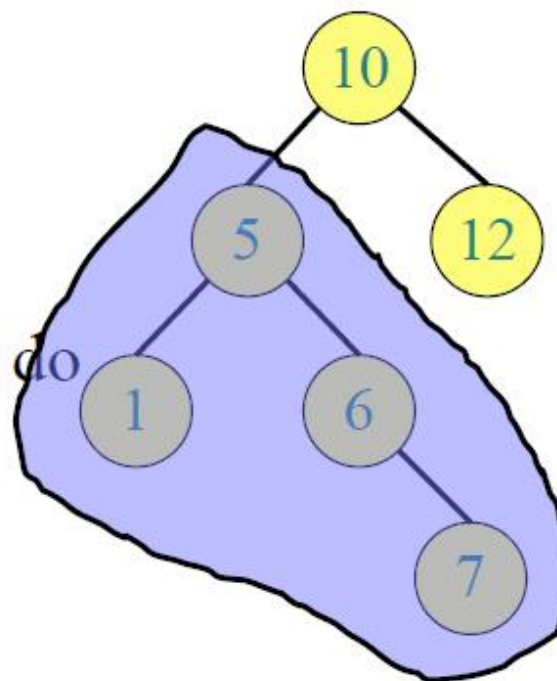
- Otherwise

$y \leftarrow p[x]$

While $y \neq \text{NIL}$ and $x = \text{right}[y]$ do

- $x \leftarrow y$
- $y \leftarrow p[y]$

Return y



next-larger(5)

next-larger(7)

最小节点



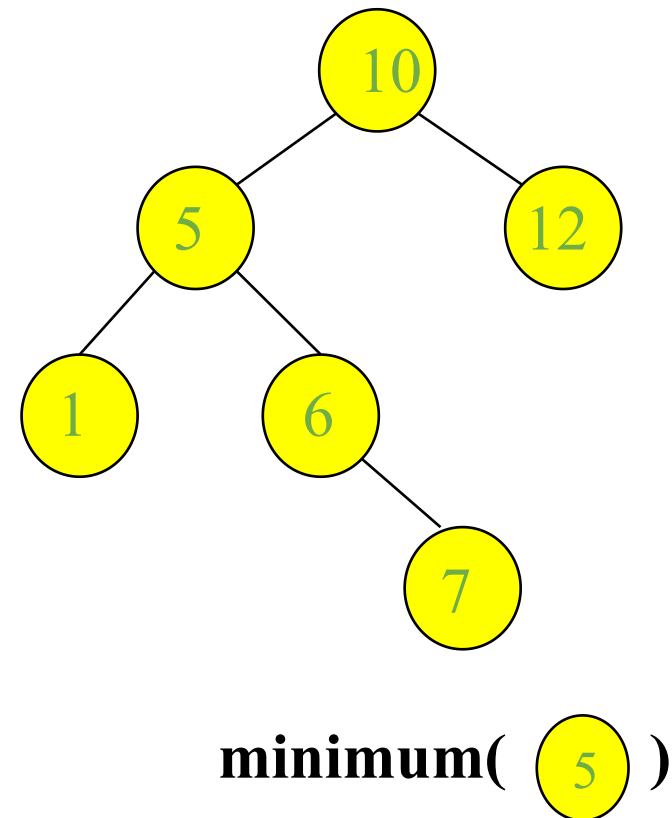
找最小节点：找出当前节点x为根节点的子树的最小节点

Minimum(x)

- While $\text{left}[x] \neq \text{NIL}$ do

$x \leftarrow \text{left}[x]$

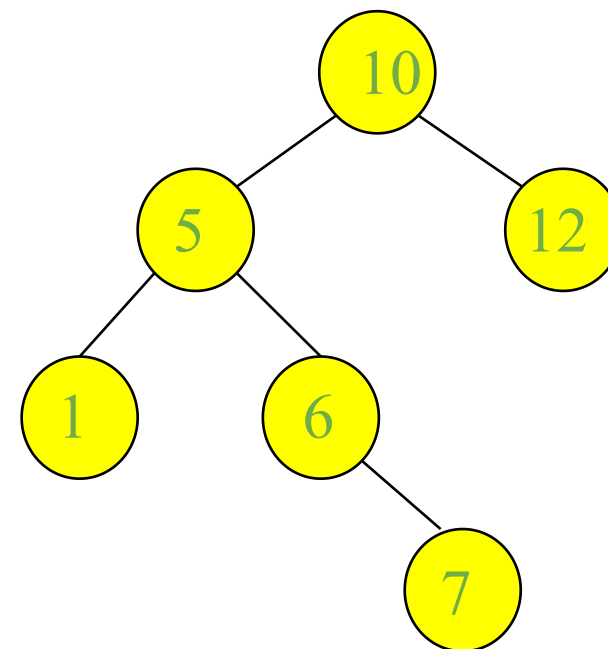
- Return x



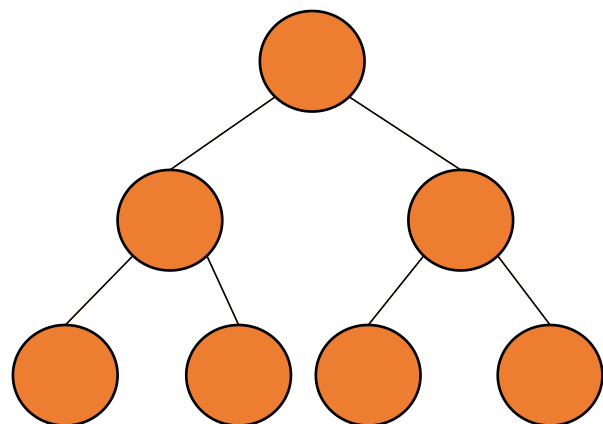
分析



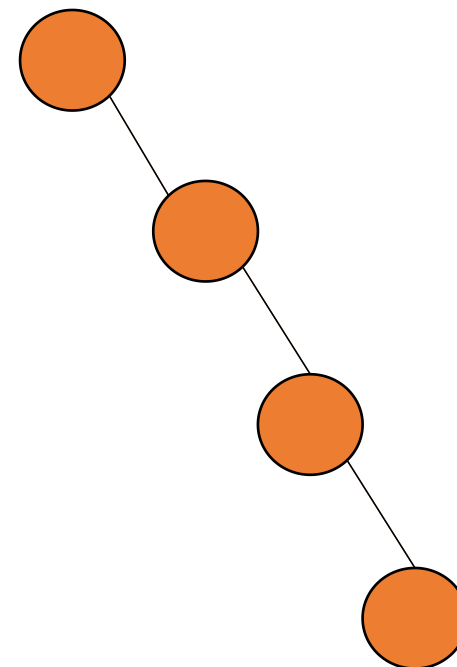
- 我们已经看到了插入、搜索、最小值等操作。
- 这些操作需要多长时间？
- $O(\text{height})$



二叉搜索树



一颗好的二叉搜索树，
高度为 $\log n$

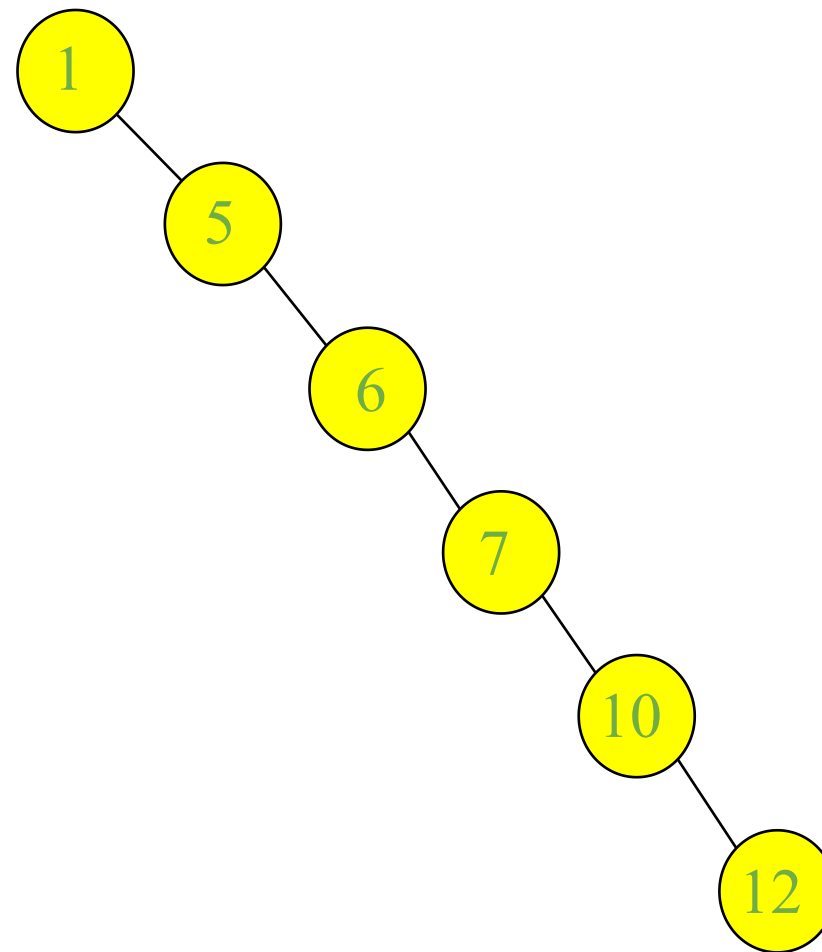


不好的二叉搜索树高度可
能为 n

分析



- 对于跑道预订系统的操作最坏情况仍然是 $\Theta(n)$ 。



分析



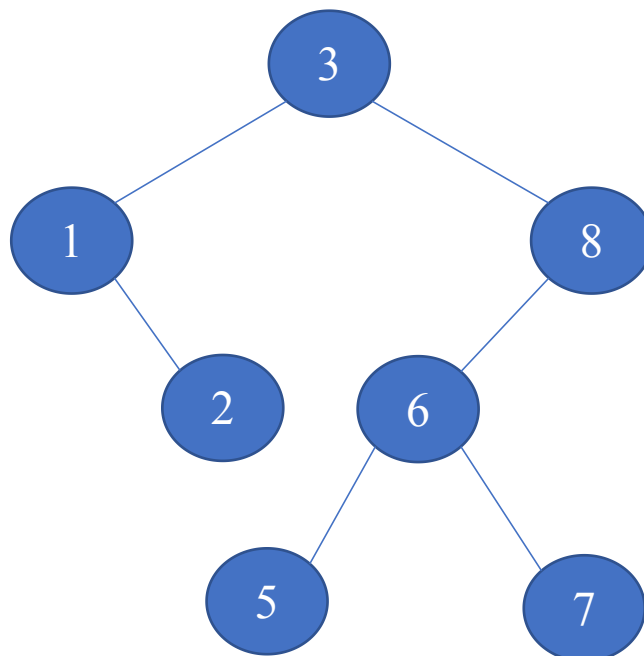
- 最坏情况: $\Theta(\text{height}) \rightarrow$ 树的高度很重要
- 当插入完 n 个元素后, 可能的最差二叉搜索树的高度是 $\Theta(n)$
- 如何避免最差情况?
- 在插入一组数据前, 先对其进行随机打乱。
- 已经证明, 若将 N 个不同的键以随机顺序插入, 则树的期望高度约为 $1.39 \lg N$

二叉搜索树排序



- 给定一个数组A，构建一个针对A的二叉搜索树 ($\Omega(n\log n)$)
- 进行中序遍历

3	1	8	2	6	7	5
---	---	---	---	---	---	---



时间复杂度



- 给定一个数组A，构建一个针对A的二叉搜索树
($\Omega(n \log n)$)
- 进行中序遍历 ($O(n)$)

与快速排序的关系



- 二叉搜索树排序中的比较与快速排序中的比较相同。

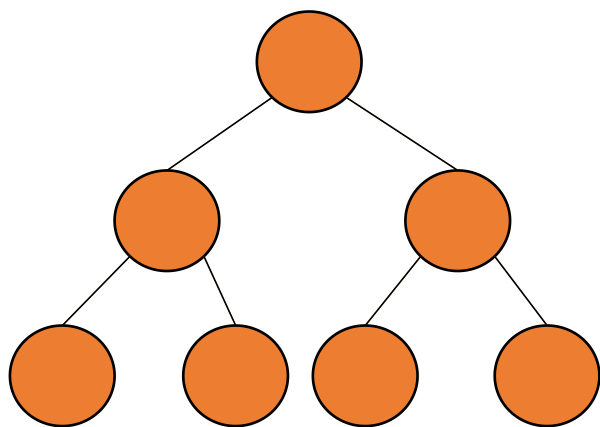
3	1	8	2	6	7	5
---	---	---	---	---	---	---

- 我们可以随机化二叉搜索树排序。
- 随机化的二叉搜索树排序具有与随机化快速排序相同的时间复杂度。

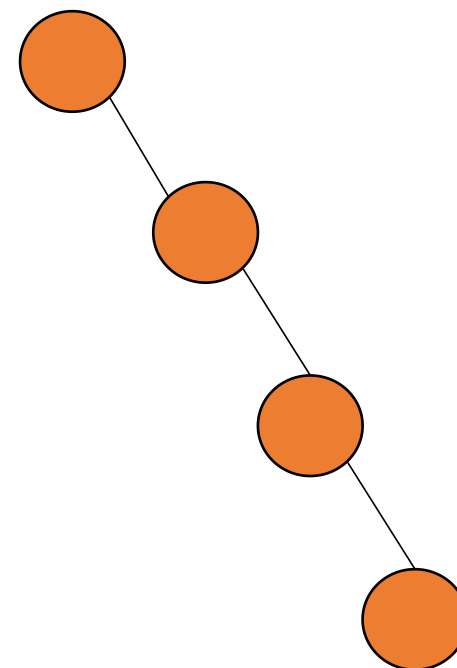
二叉搜索树的优化问题分析



- 不好的二叉搜索树与好的二叉搜索树有什么区别？



好的二叉搜索树



不好的二叉搜索树

平衡二叉搜索树的策略



- 给所有的节点增加一些额外的信息
- 给每个本地节点的信息定义一个不变式
- 证明每个本地节点的不变式可以保证树的高度为 $O(\log n)$
- 设计算法来维持额外的节点信息和不变式



平衡搜索树



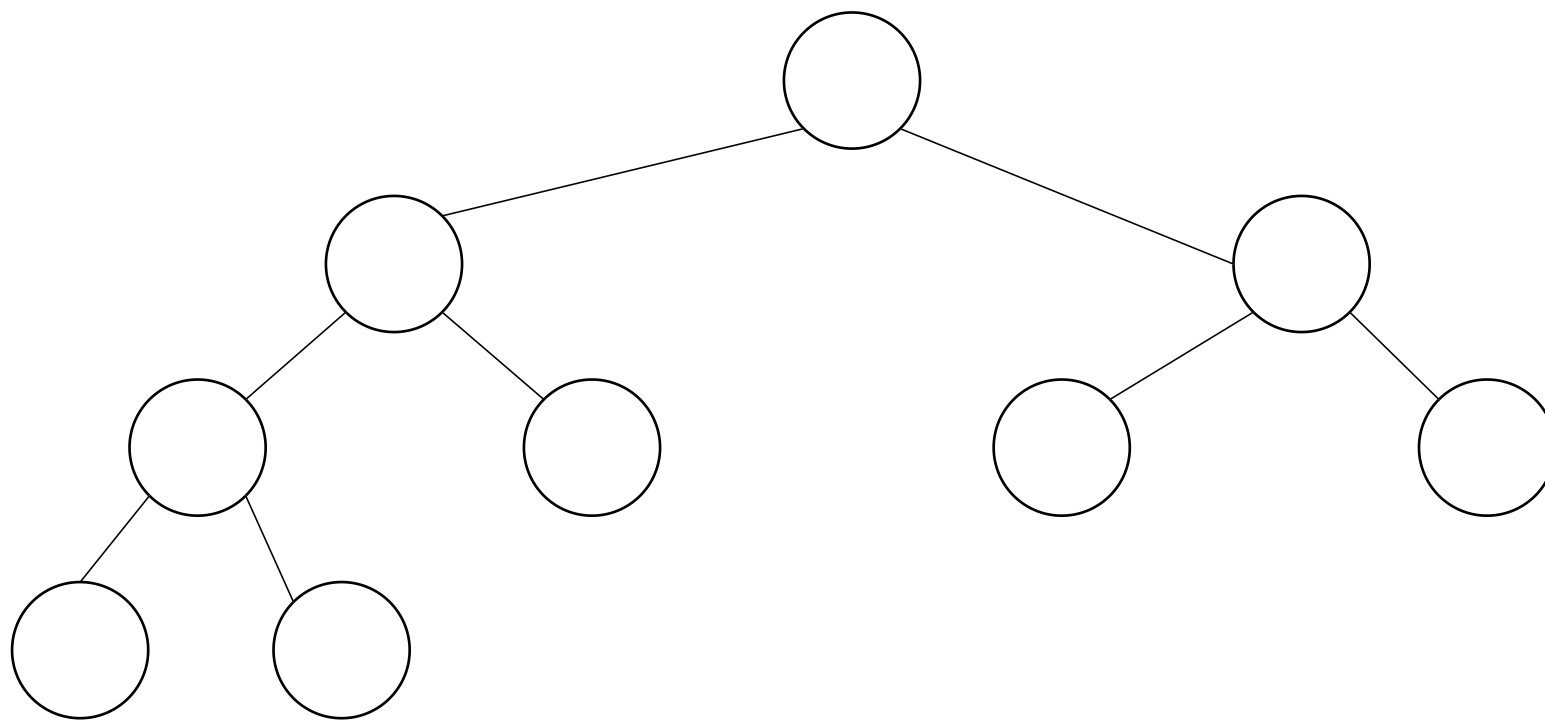
- AVL树 (Adelson-Velsii和Landis于1962年提出)
- 红黑树 (参见CLRS第13章)
- 伸展树 (Sleator和Tarjan于1985年提出)
- 替罪羊树 (Galperin和Rivest于1993年提出)
- Treap (Seidel和Aragon于1996年提出)

好的二叉搜索树的特点



- 二叉搜索树长成完全二叉树好吗?

- 好!

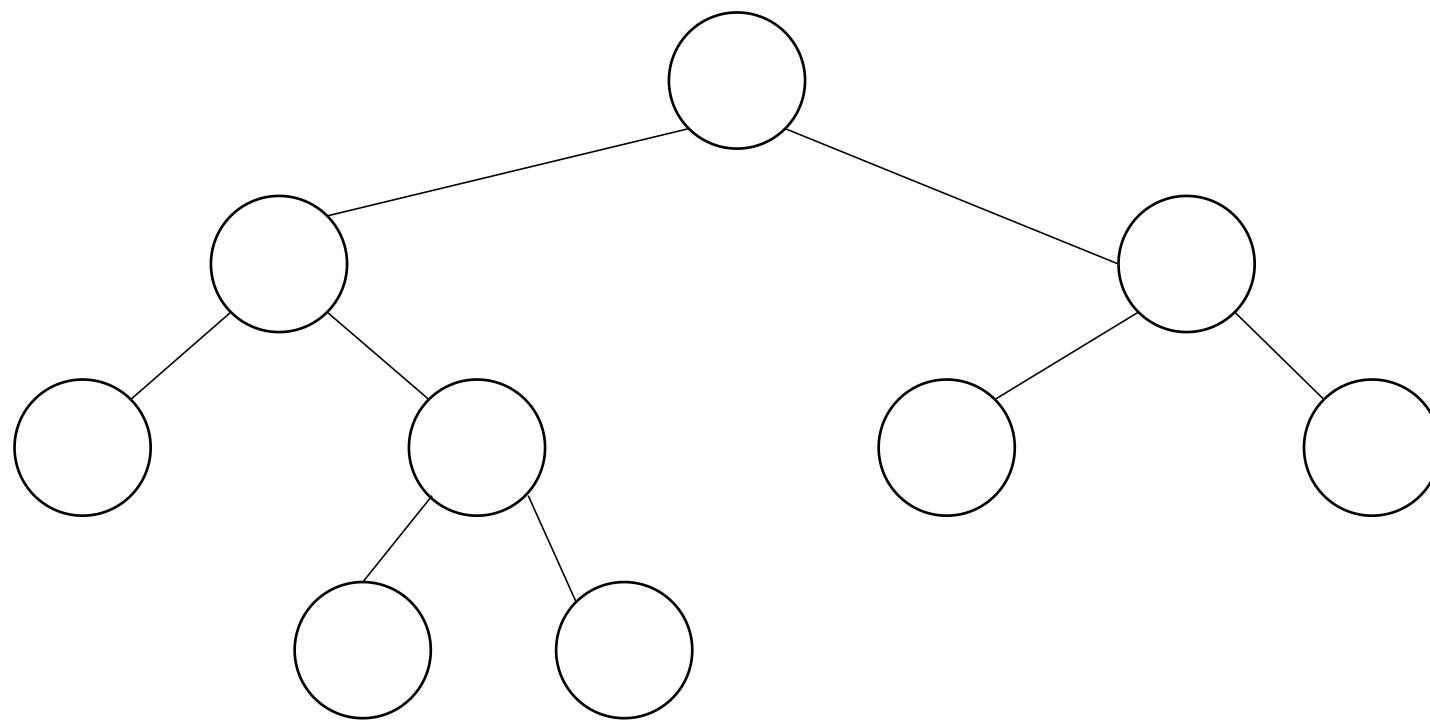


好的二叉搜索树的特点



● 下面二叉搜索树好吗?

● 好!

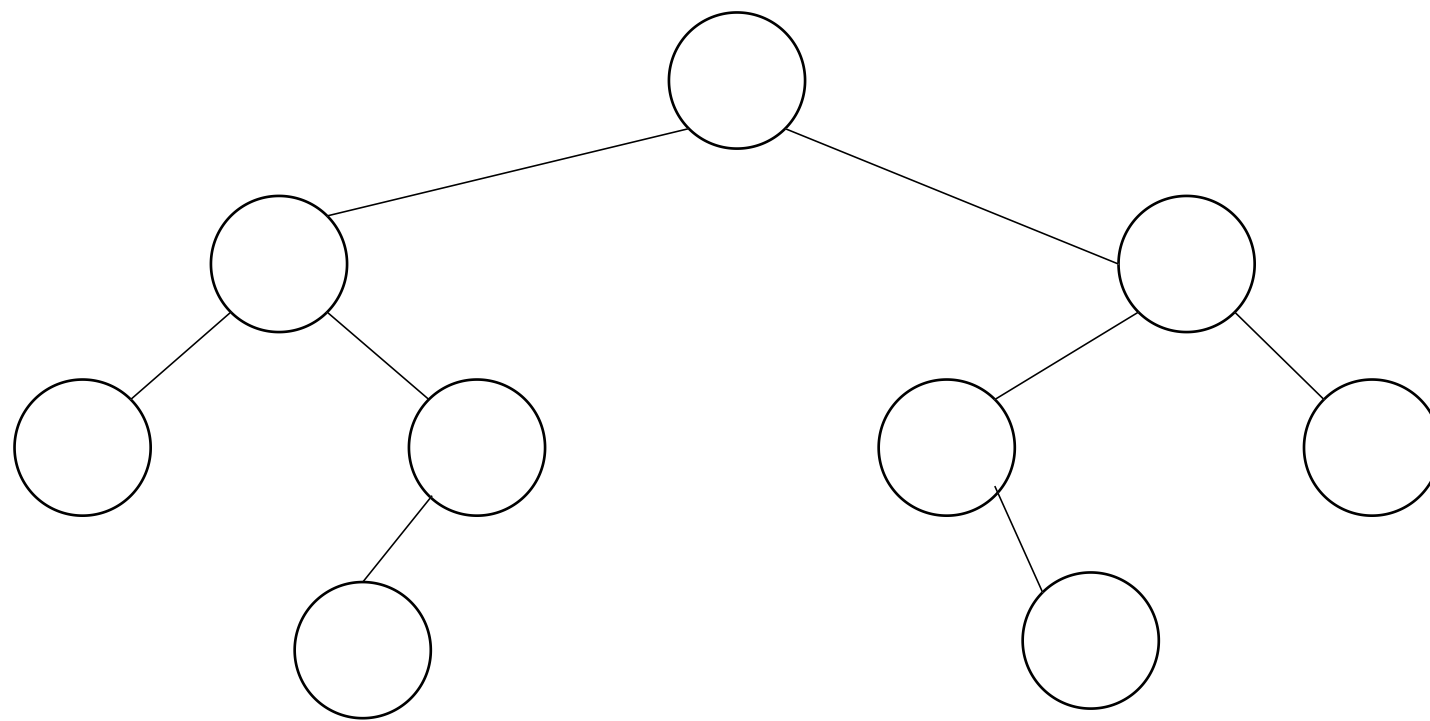


好的二叉搜索树的特点



● 下面二叉搜索树好吗?

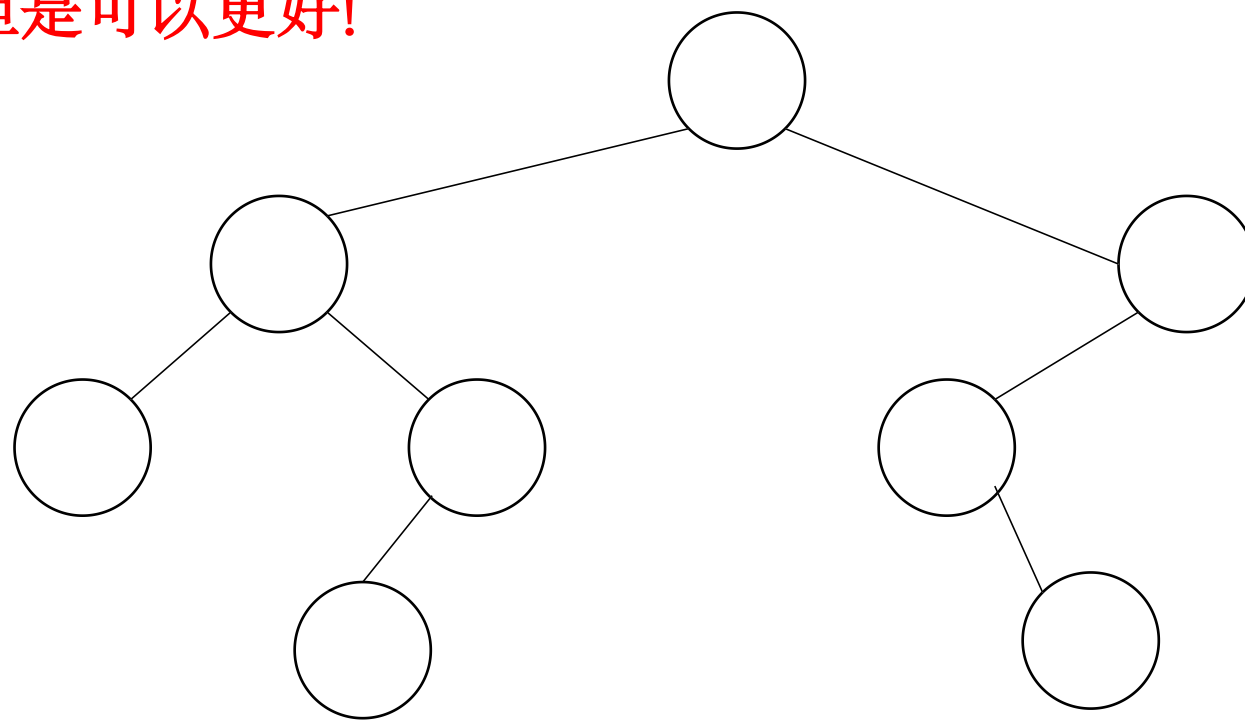
● 好!



好的二叉搜索树的特点



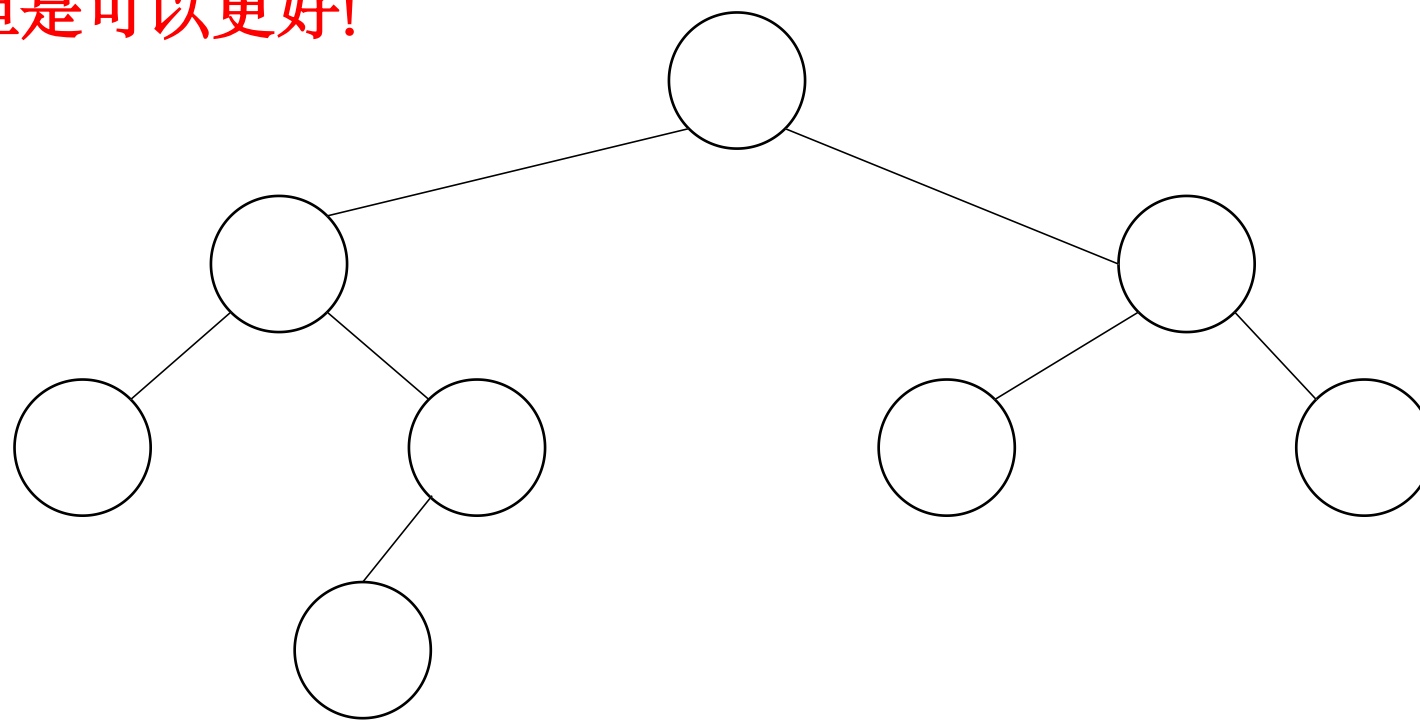
- 下面二叉搜索树好吗?
- 还好, 但是可以更好!



好的二叉搜索树的特点



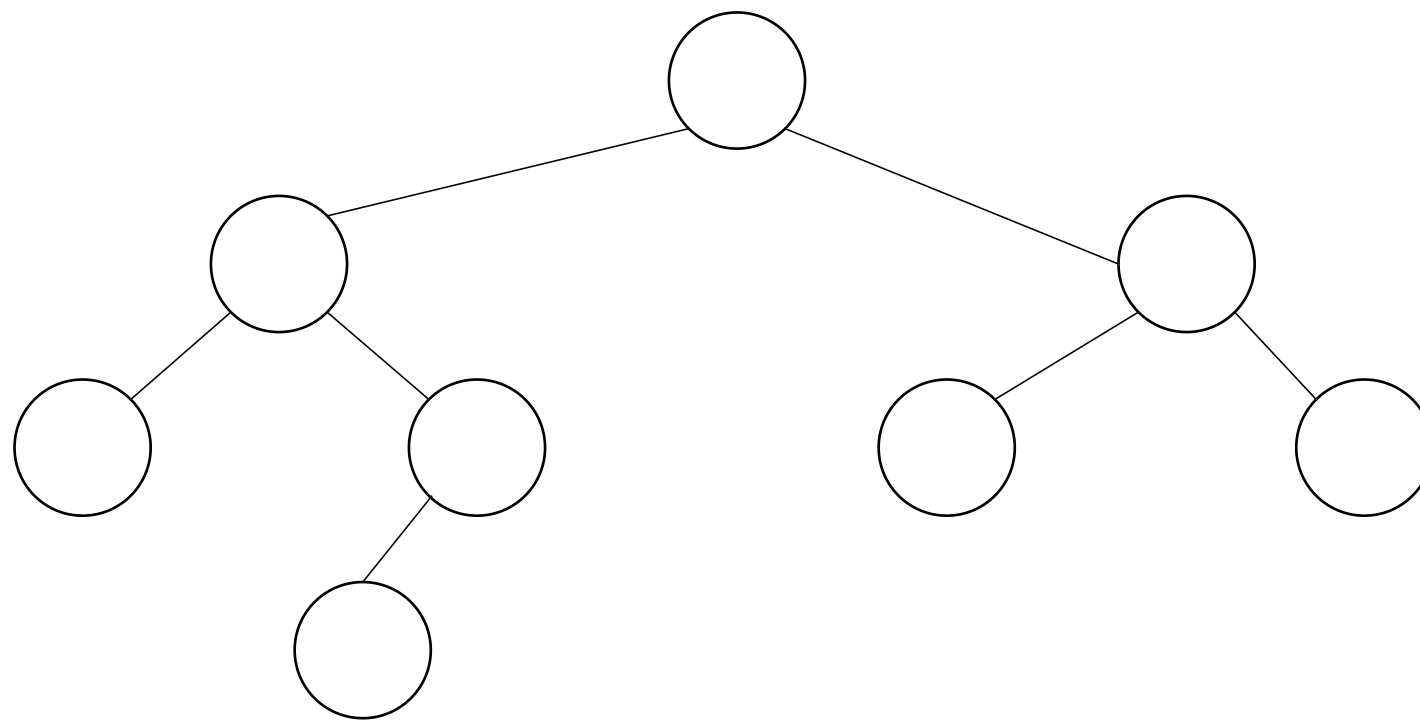
- 下面二叉搜索树好吗?
- 还好, 但是可以更好!



好的二叉搜索树的特点



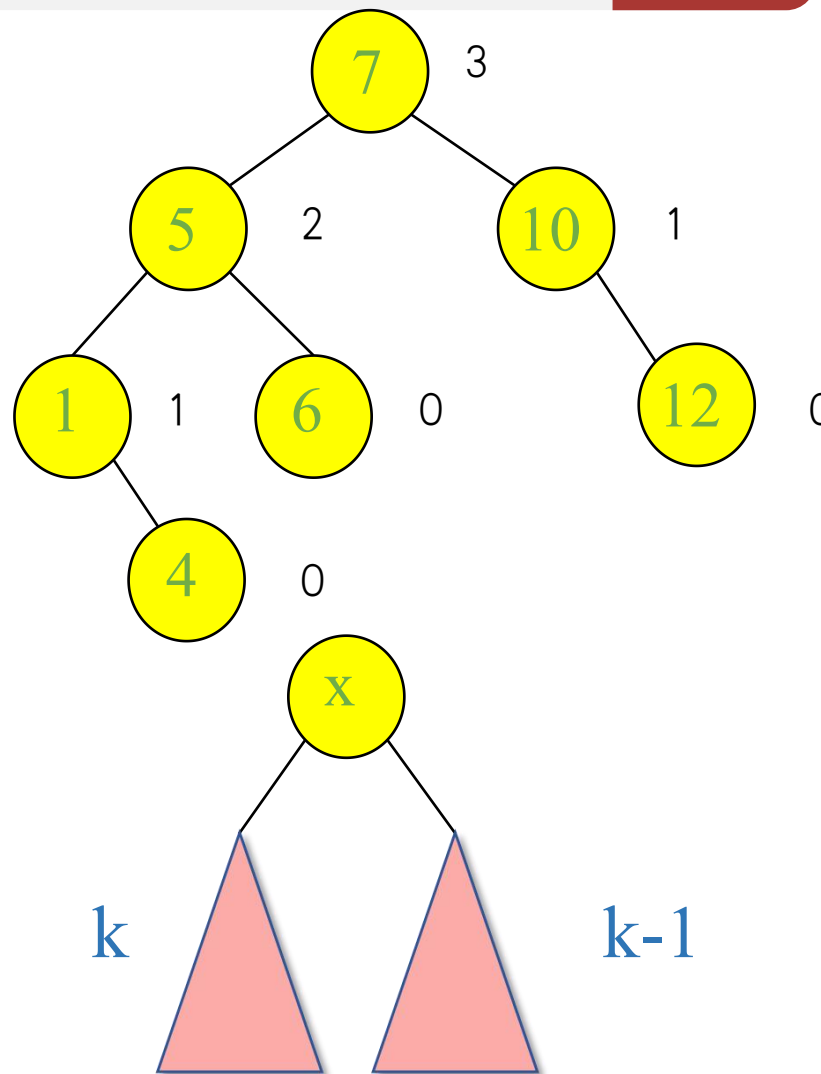
- 任意一个节点的左子树和右子树深度相当



AVL树：定义



- **信息：** 对于每一个节点，维护它的高度（“增加物”）
 - 叶节点高度为0
 - 空节点高度为-1
- **不变式：** 对于每一个节点，左子树与右子树的高度差为1



AVL树的高度为 $O(\log n)$

不变式：对于每个节点x，其左子树和右子树的高度差最多为1。

➤ 设 n_h 为高度为h的AVL树的最小节点数。

➤ 我们有 $n_h \geq 1 + n_{h-1} + n_{h-2}$

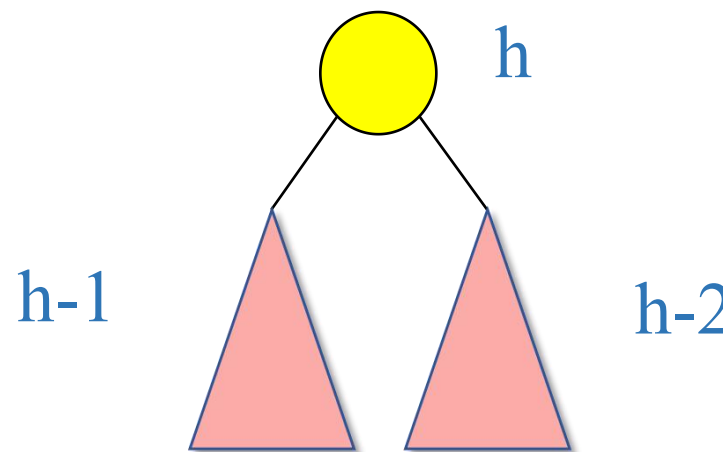
$$n_h > 2n_{h-2}$$

$$n_h > 2^{h/2}$$

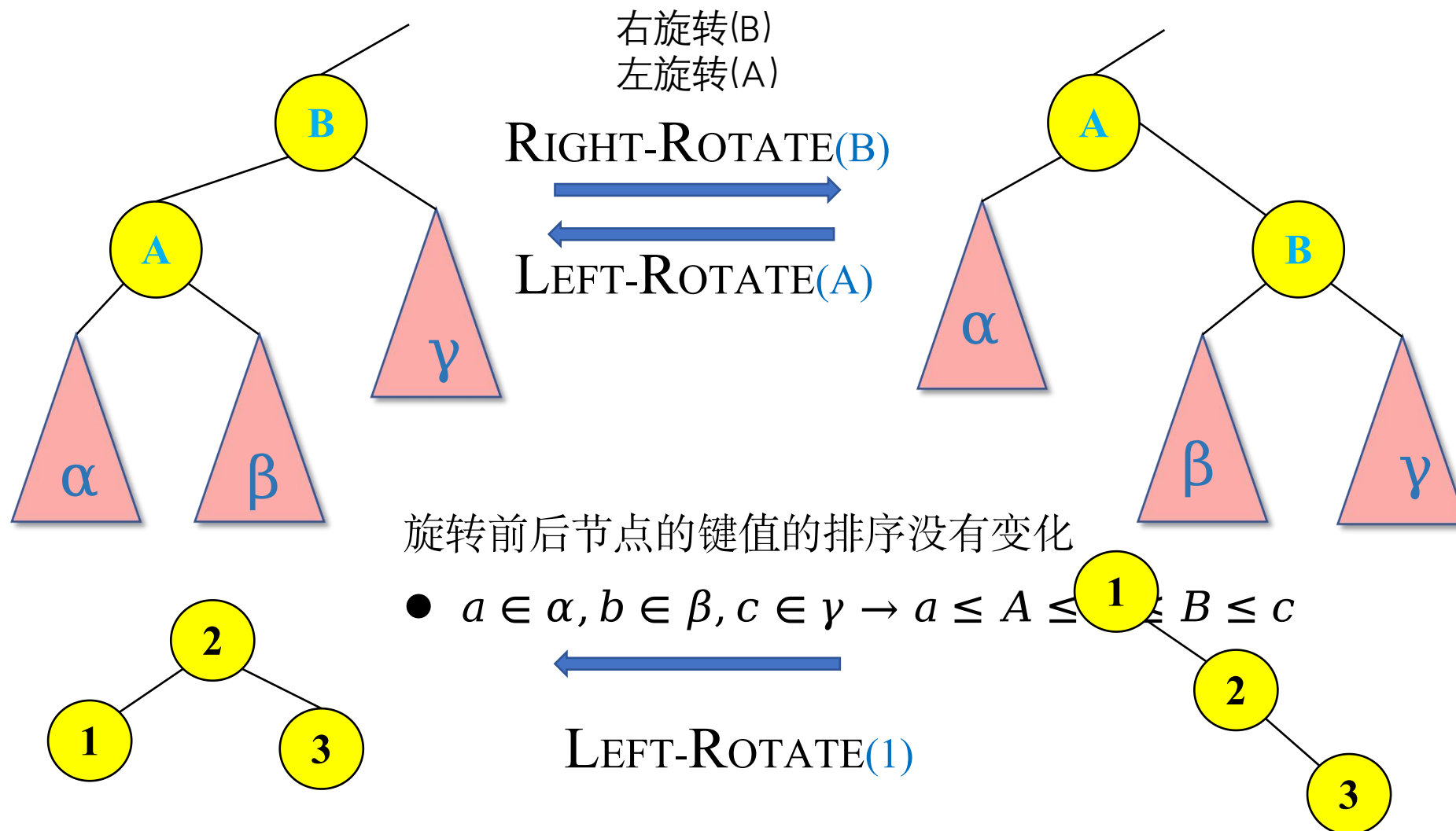
$$h < 2 \log n_h$$

➤ 常数“2”可以得到改进

每个节点满足不变式的话，上面证明了整棵树的高度必然是 $O(\log n)$ ，下一步是怎样来维护每个节点的不变式？



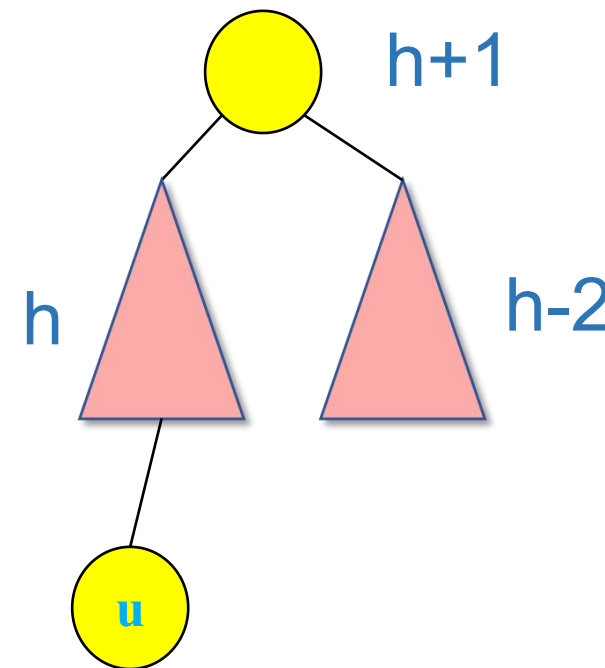
旋转



插入



- 插入一个新的节点u可能导致不平衡
- 为了解决这个问题，你需要沿着树向上移动，恢复平衡。
- 同样的问题和解决方案也适用于删除节点时的情况。



平衡



- 使X节点是最低的违反属性的节点

先修复X为根节点的子树，再一步步往上修复

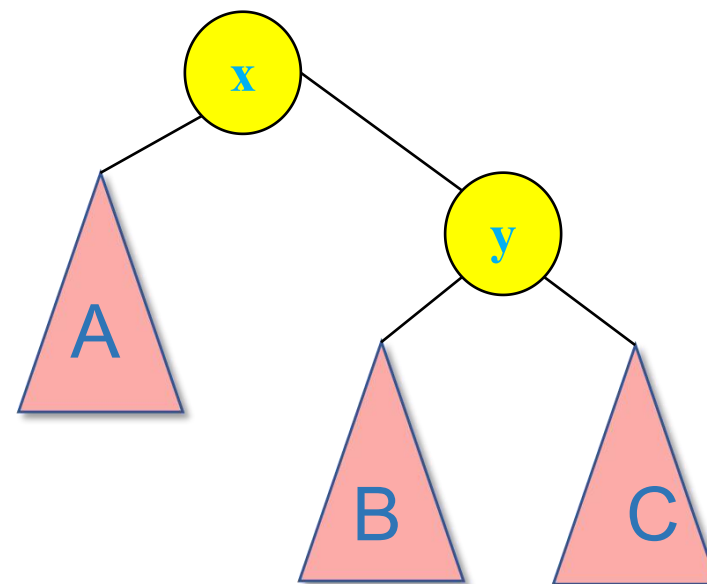
- 右高：假设x右子树比左子树要高

- 可能出现3种情况

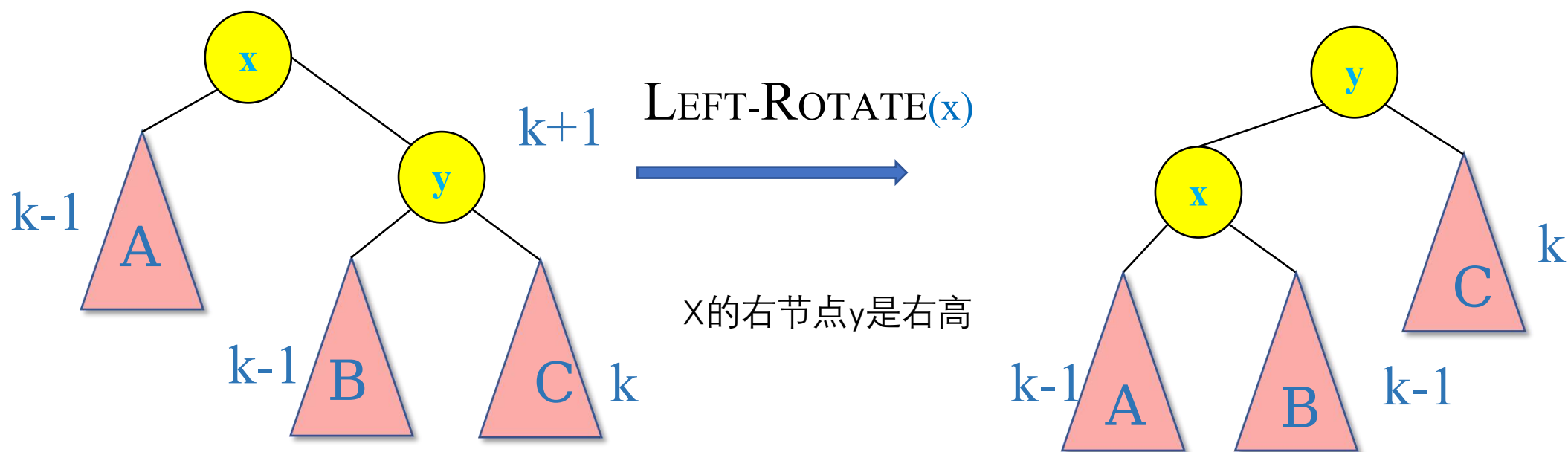
X的右节点y是右高

X的右节点y是平衡的

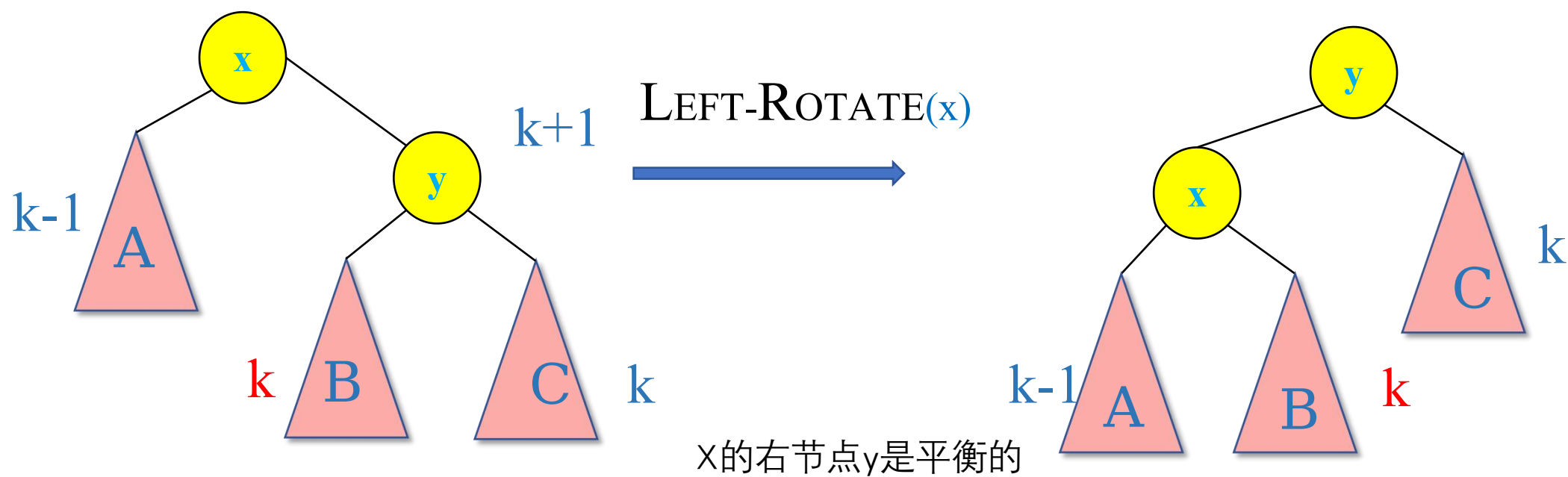
X的右节点y是左高



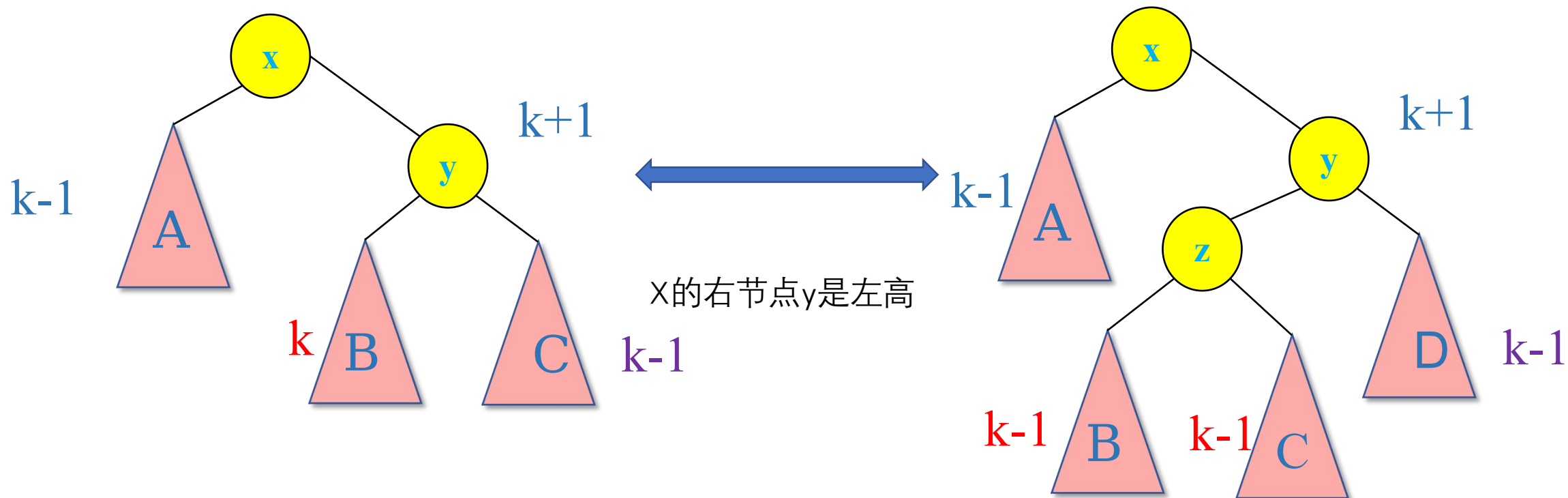
情况1: y是右重的



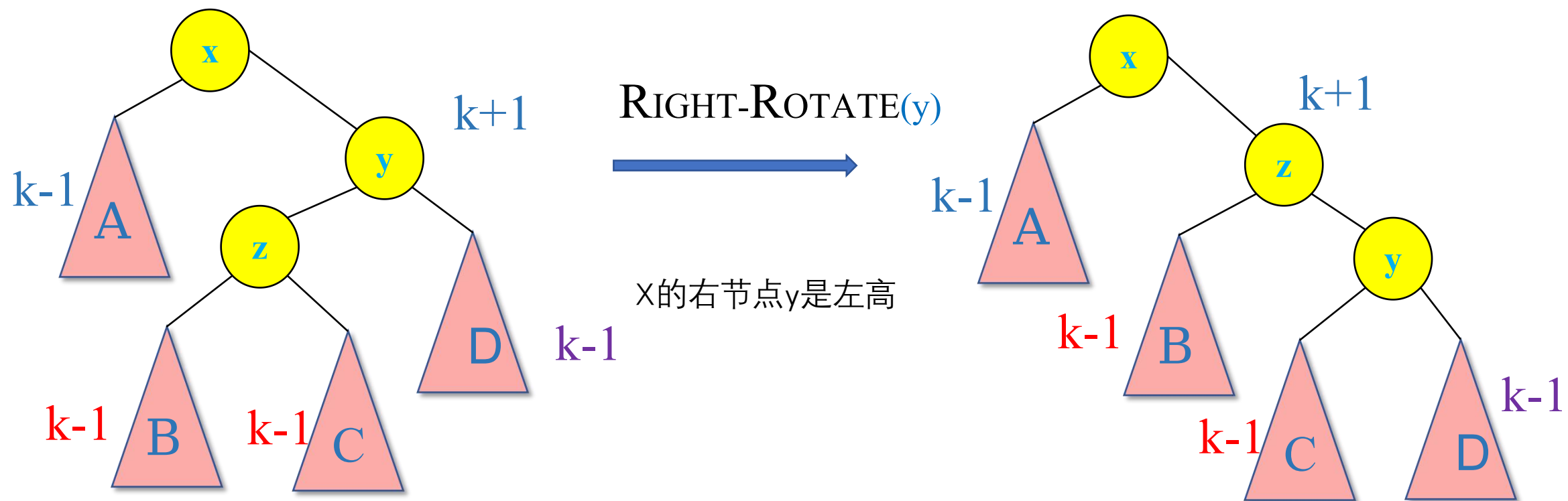
情况2: y 是平衡的



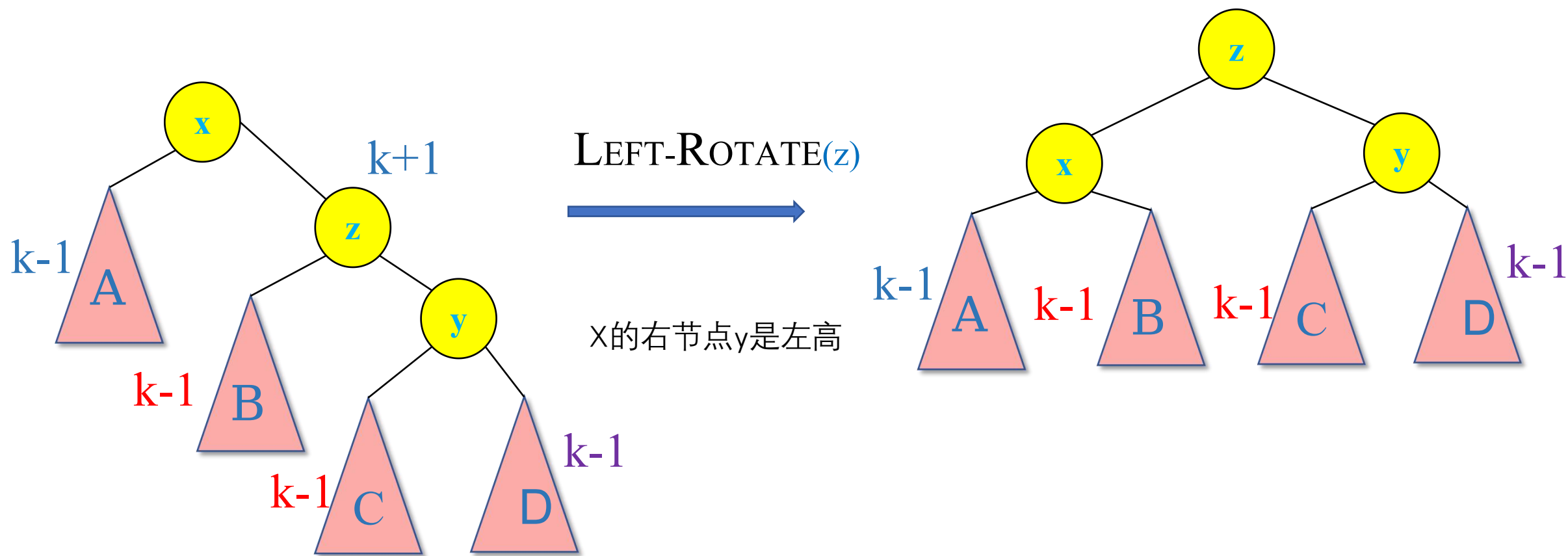
情况3: y 是左重的



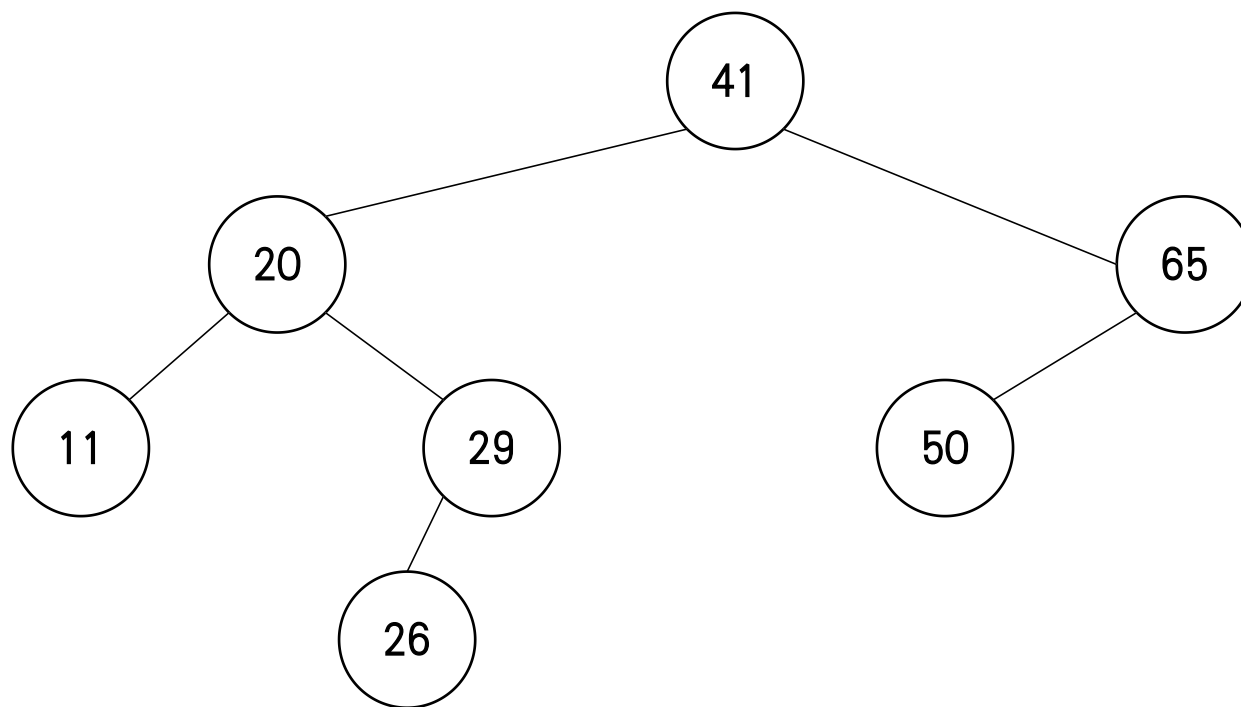
情况3: y是左重的



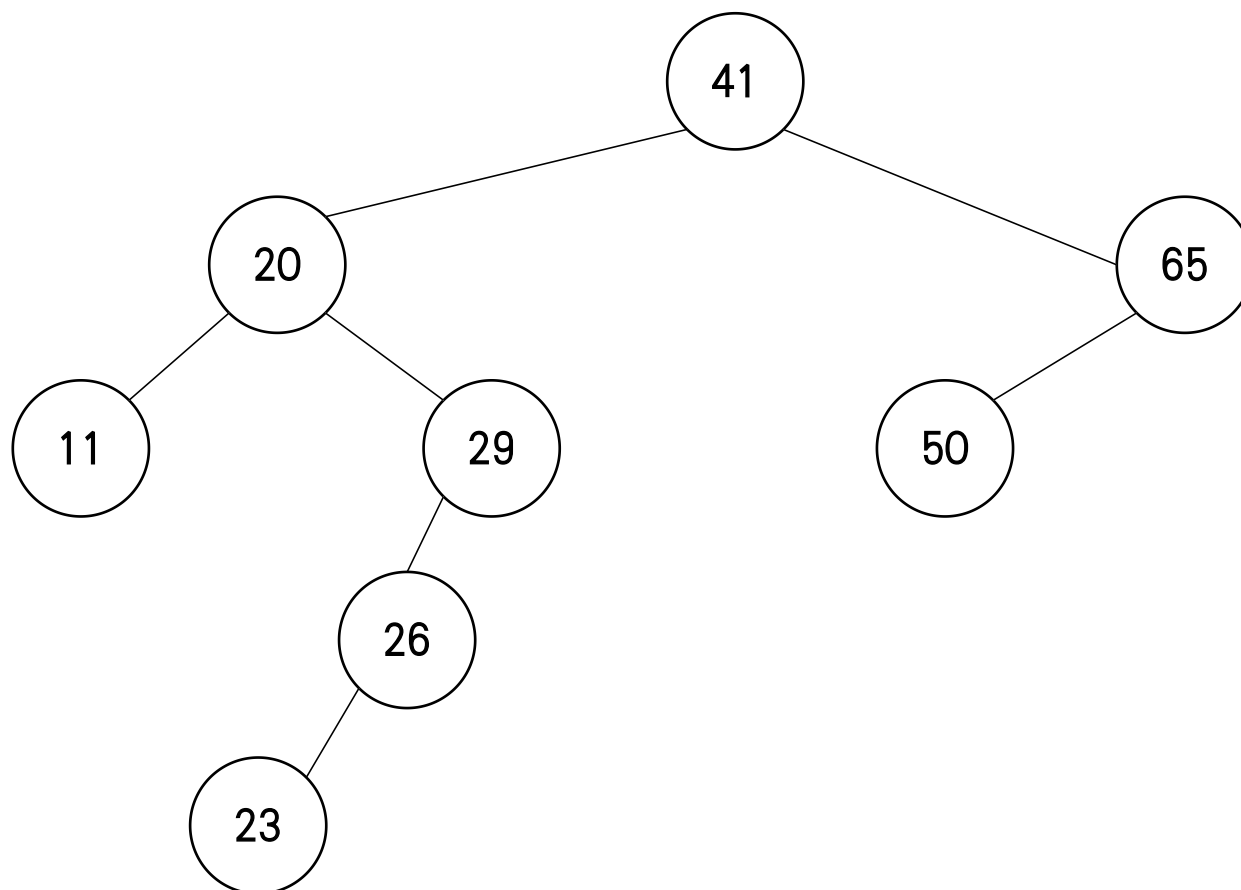
情况3: y是左重的



AVL树插入示例

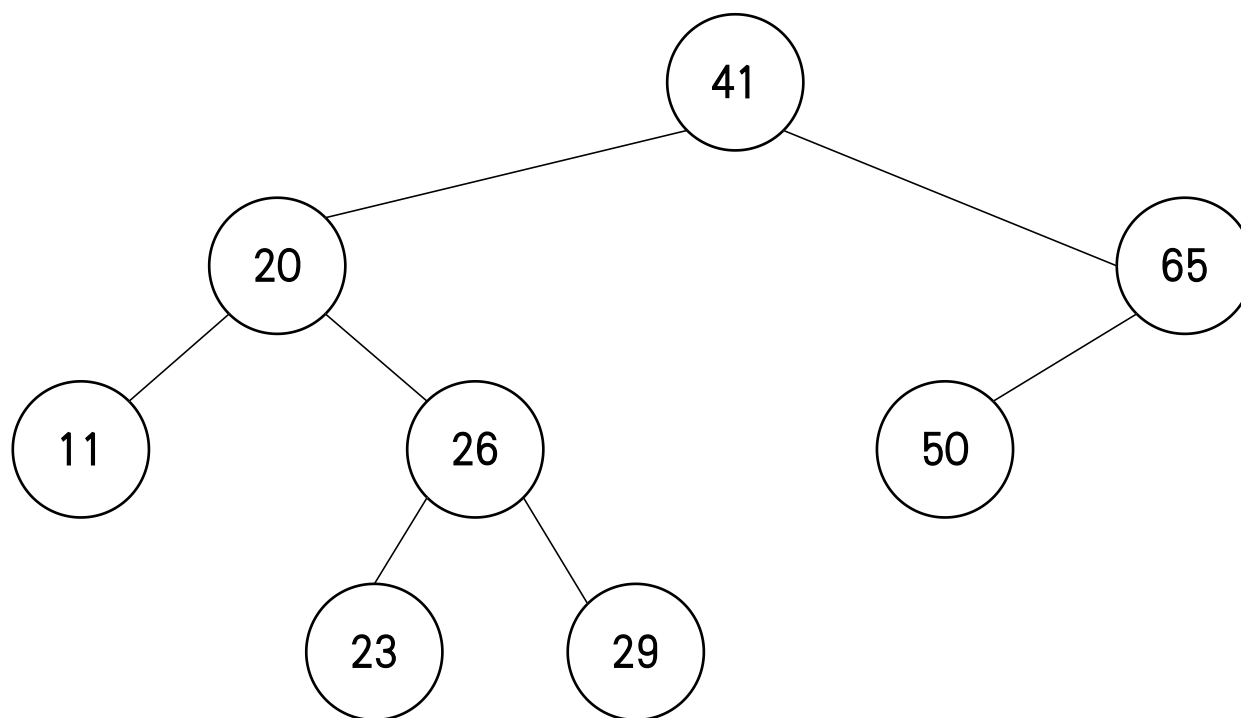


AVL树插入



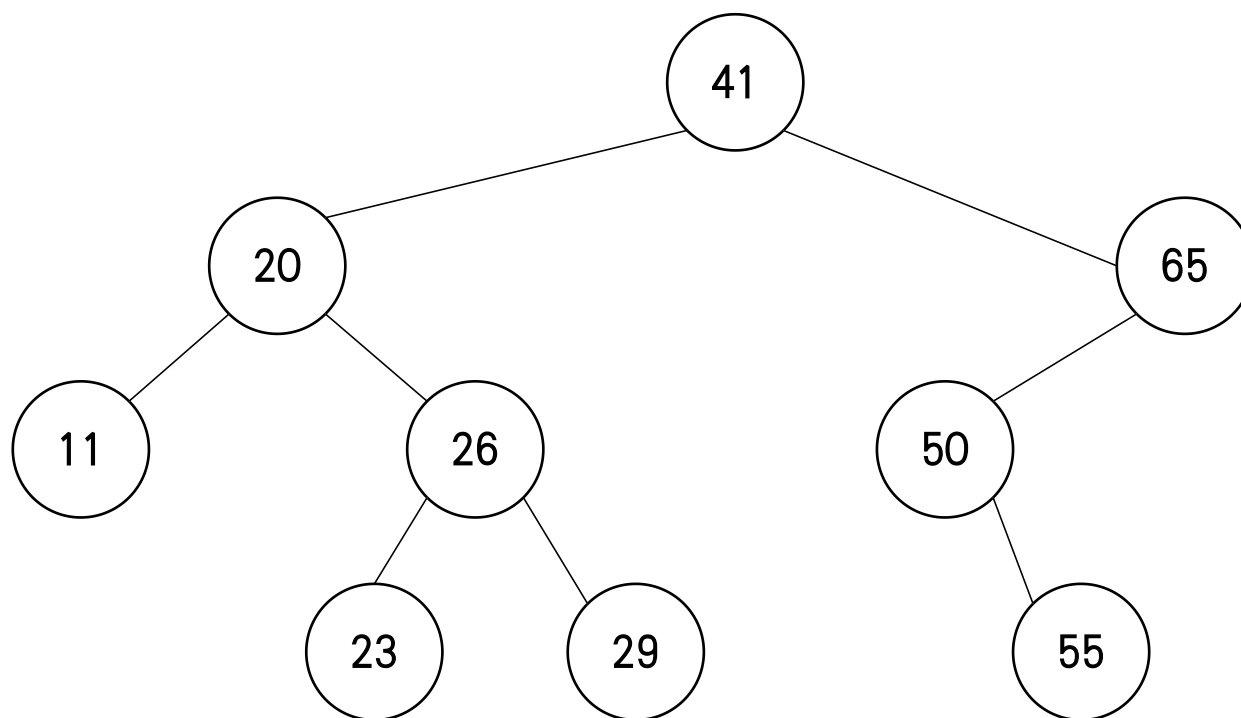
插入 23

AVL树插入



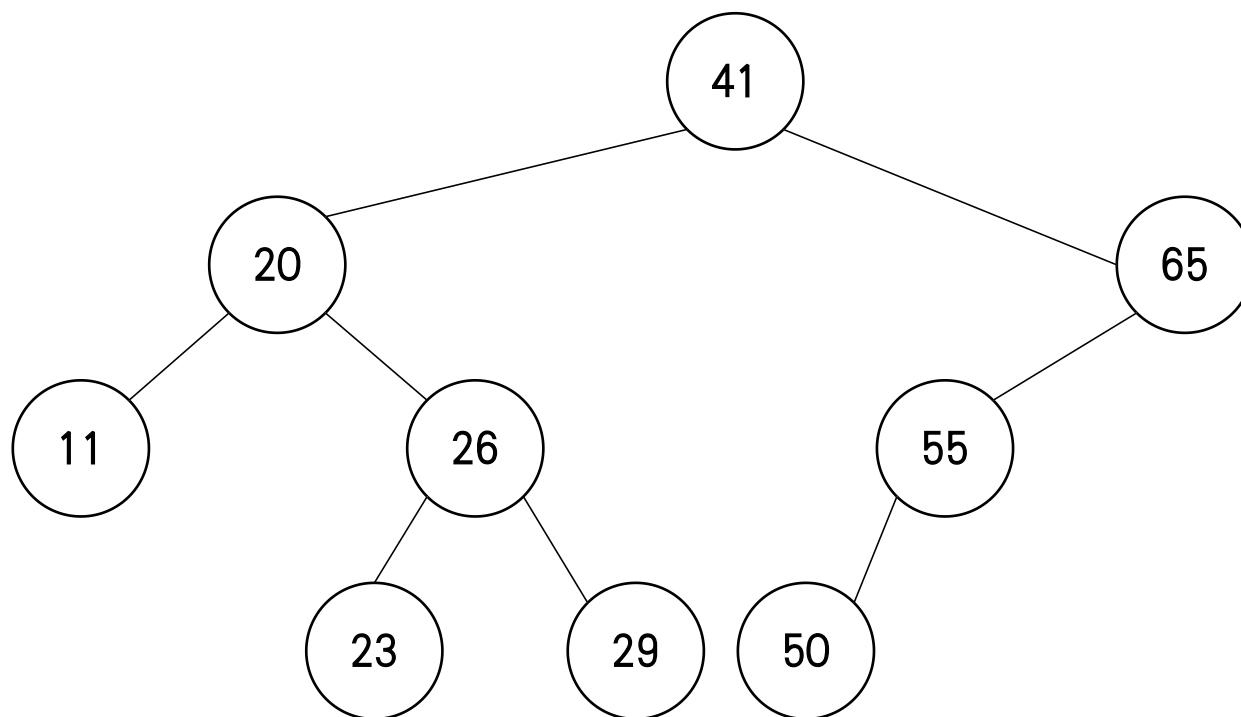
插入 23

AVL树插入



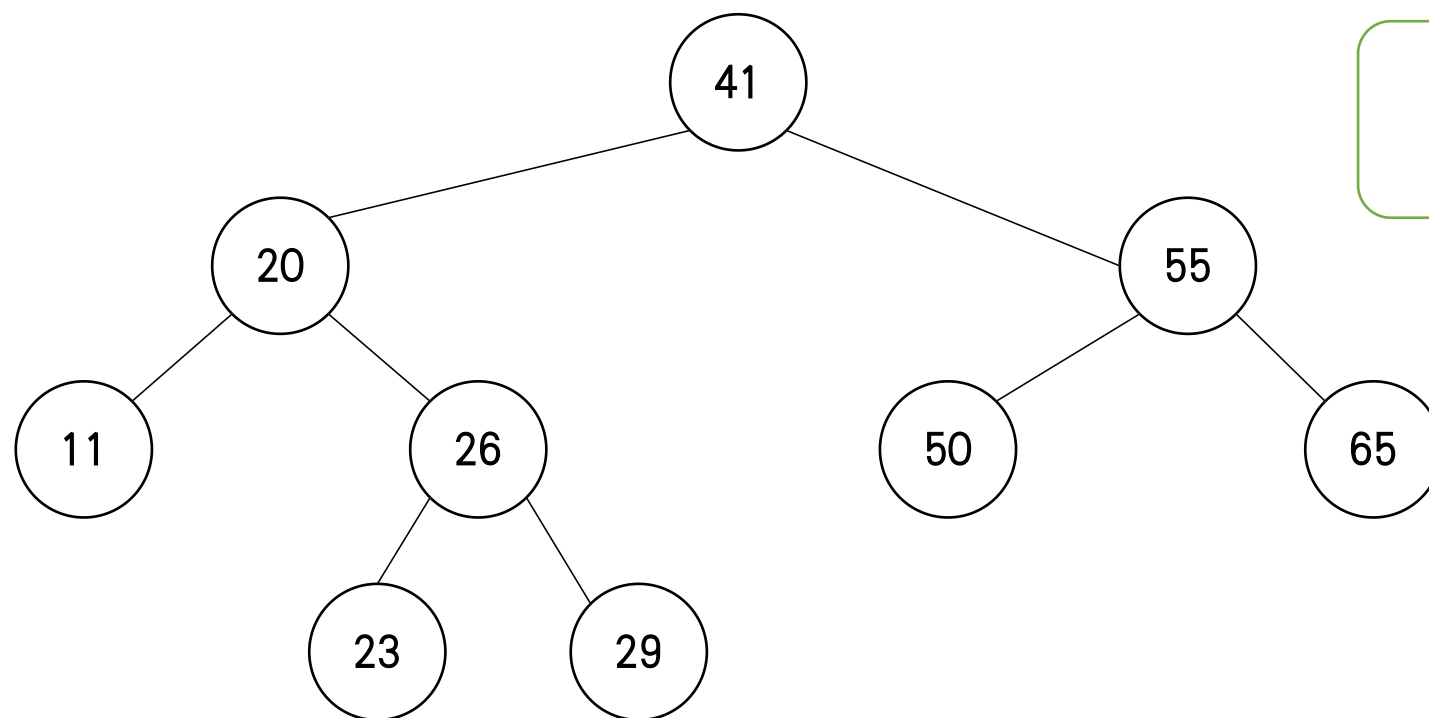
插入 55

AVL树插入



插入 55

AVL树插入



插入 55

结论

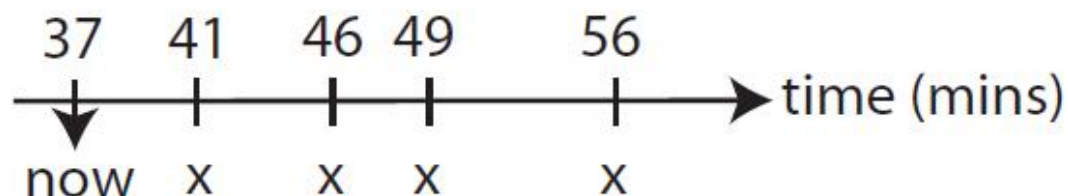


- 可以每次插入时在 $O(\log n)$ 时间内维护AVL树。
- 搜索等操作需要 $O(\log n)$ 时间。

AVL树 ——跑道预约系统



- 当前降落时间集合 $R = \{37, 41, 46, 49, 56\}$



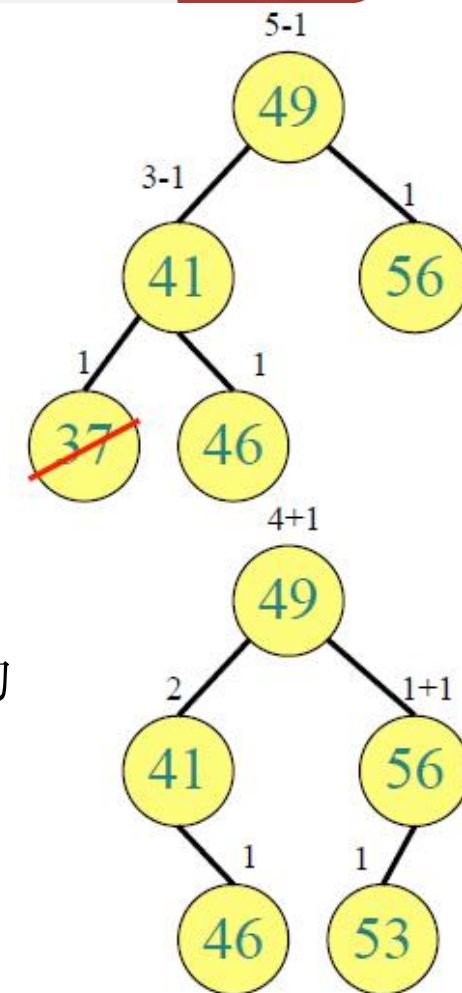
- 当一架飞机降落时，从集合中移除时间 t

$R = (41, 46, 49, 56)$

- 如果在距离 t 不到3分钟的时间内没有其他预定降落，则向集合中添加新的时间 t

44=>拒绝 (R 中的46) 53=>好的

- 删除、插入和冲突检查的时间复杂度为 $O(h)$ ，其中 h 是树的高度。



AVL树缺点分析



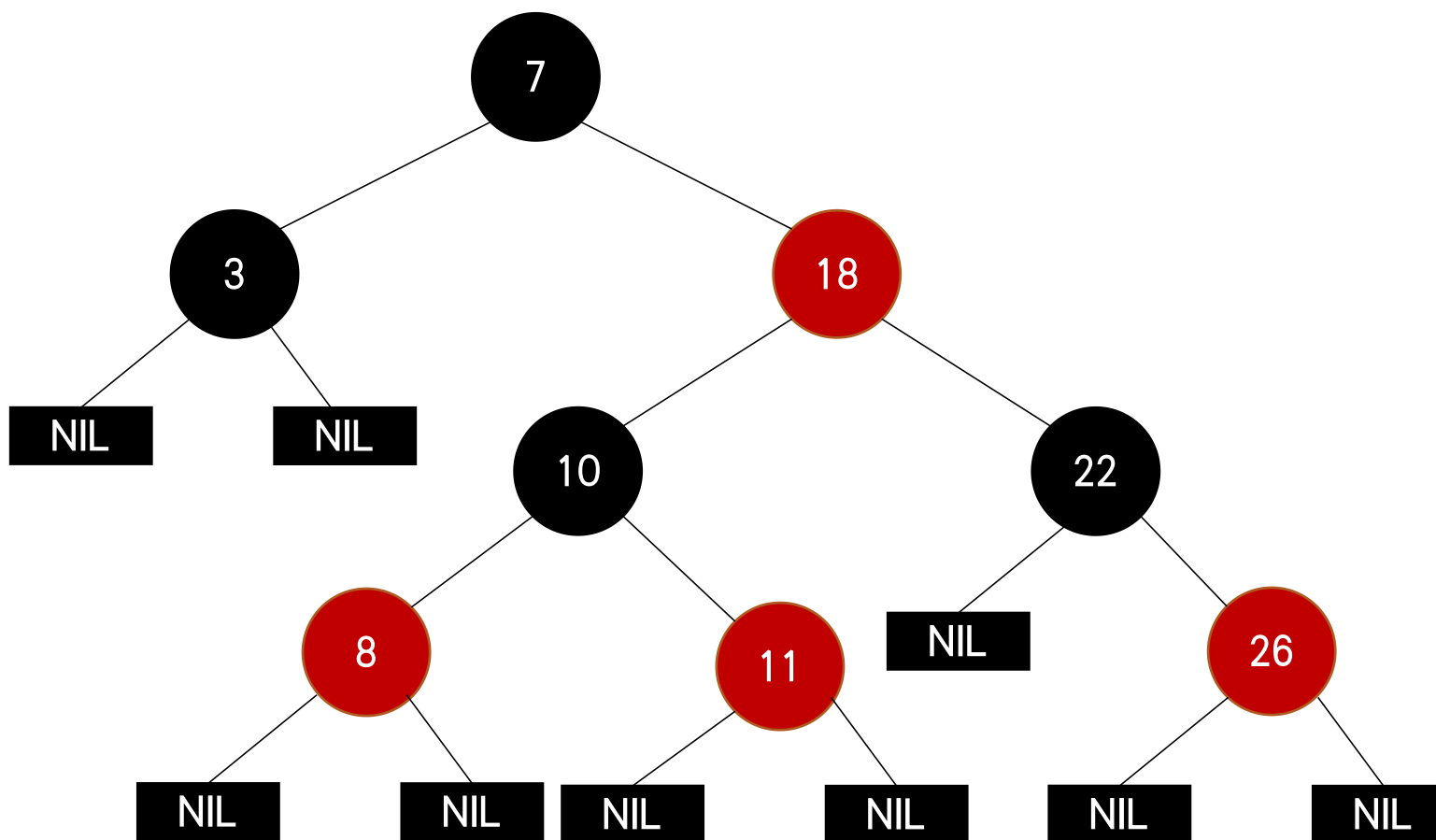
- AVL树的问题?
- 对更新太敏感

红黑树



- 带有额外颜色的BST结构，满足以下条件：
 - 1、每个节点要么是黑色，要么是红色；
 - 2、根和叶子都是黑色的，所有的叶子都是NIL；
 - 3、红色节点的父节点是黑色的；
 - 4、从节点x到其所有后代叶子节点的所有路径中包含相同数量的黑节点。

示例



红黑树



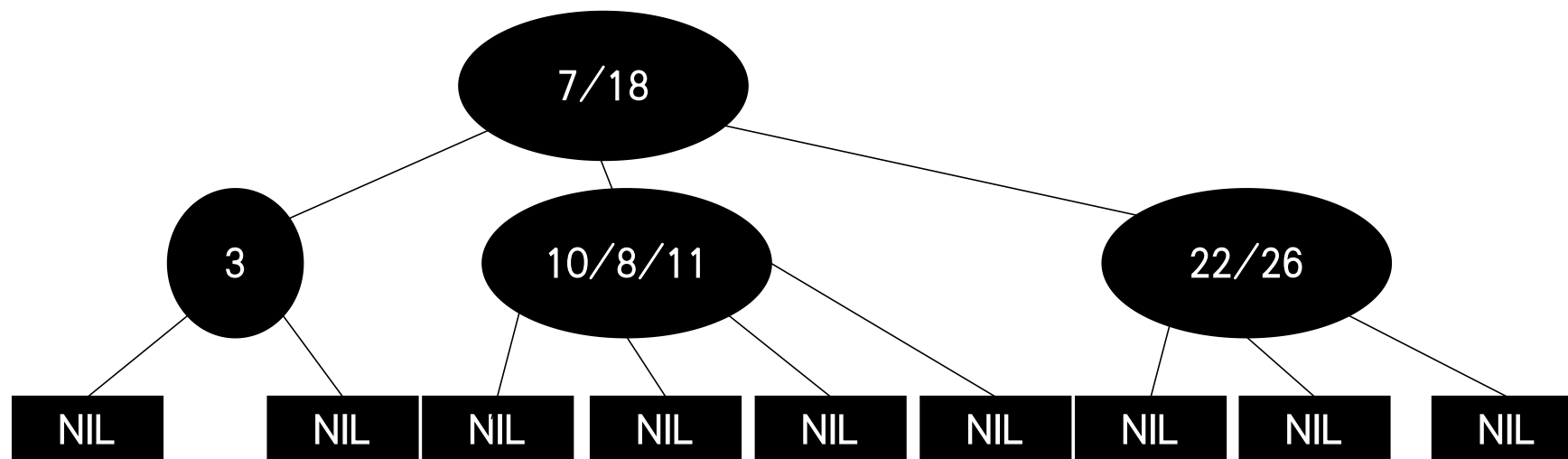
- 在红黑树中，从节点 x 到其所有后代叶子节点的所有路径中相同数量的黑节点被称为 x 的黑高度。
- 假设红黑树中有 n 个节点，则有 $n+1$ 个叶子（通过归纳法证明）。

红黑树的高度



- 红黑树的高度小于 $2\log(n+1)=O(\log n)$
- 证明：将红色节点与其黑色父节点合并。然后，树变成了一个2-3-4树，其中每个节点有2、3或4个子节点，所有叶子具有相同的深度，即黑色高度 h' 。

证明示例



红黑树的高度

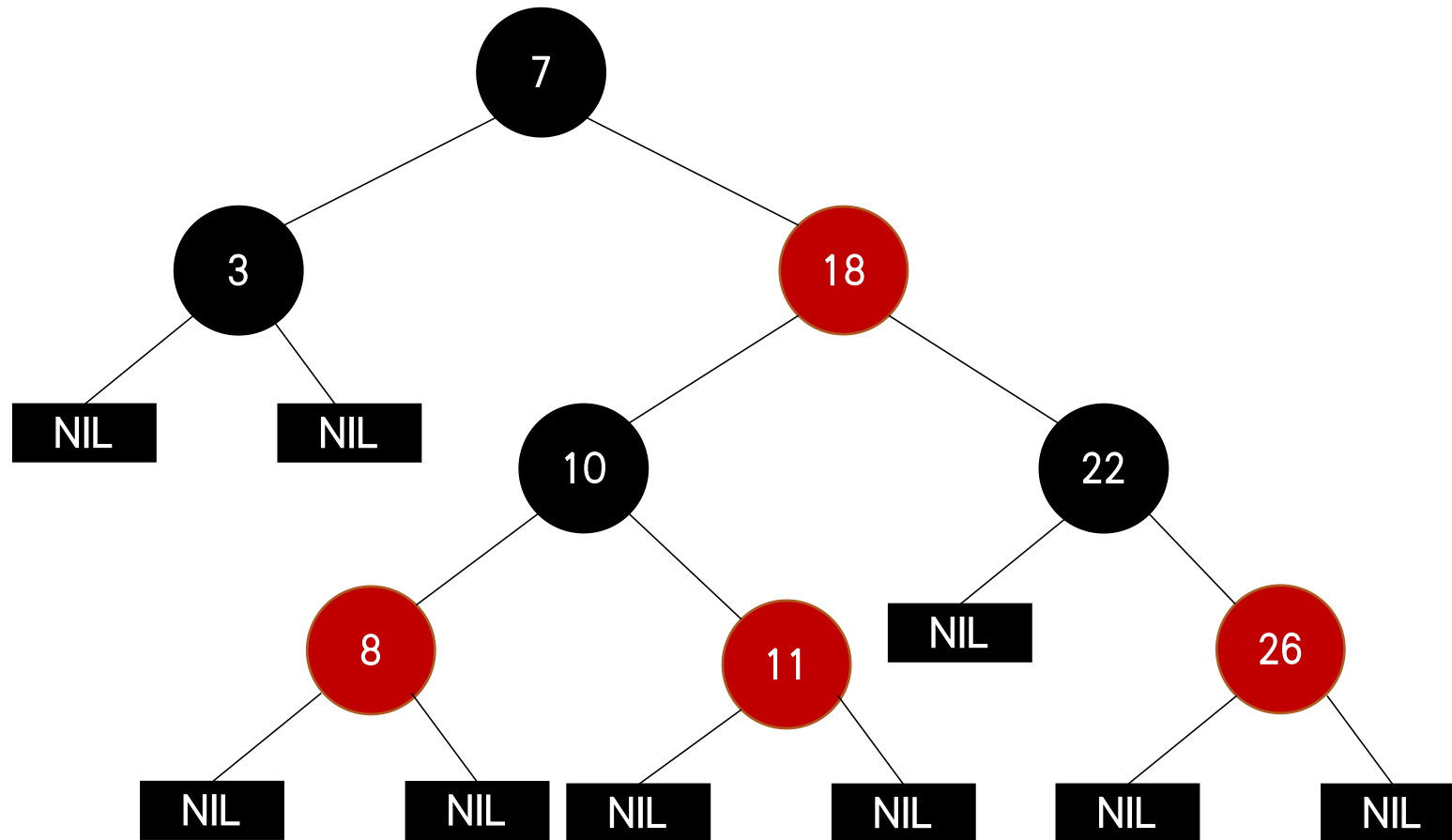


- 红黑树的高度小于 $2\log(n+1)=O(\log n)$
- 证明: $2^{h'} \leq \#leaves \leq 4^{h'} \Rightarrow 2^{h'} \leq n + 1 \Rightarrow h' \leq \log(n + 1)$
- 因此, 红黑树的高度 h 小于 $2h'$, 所以 $h \leq 2\log(n + 1)$

红黑树的搜索



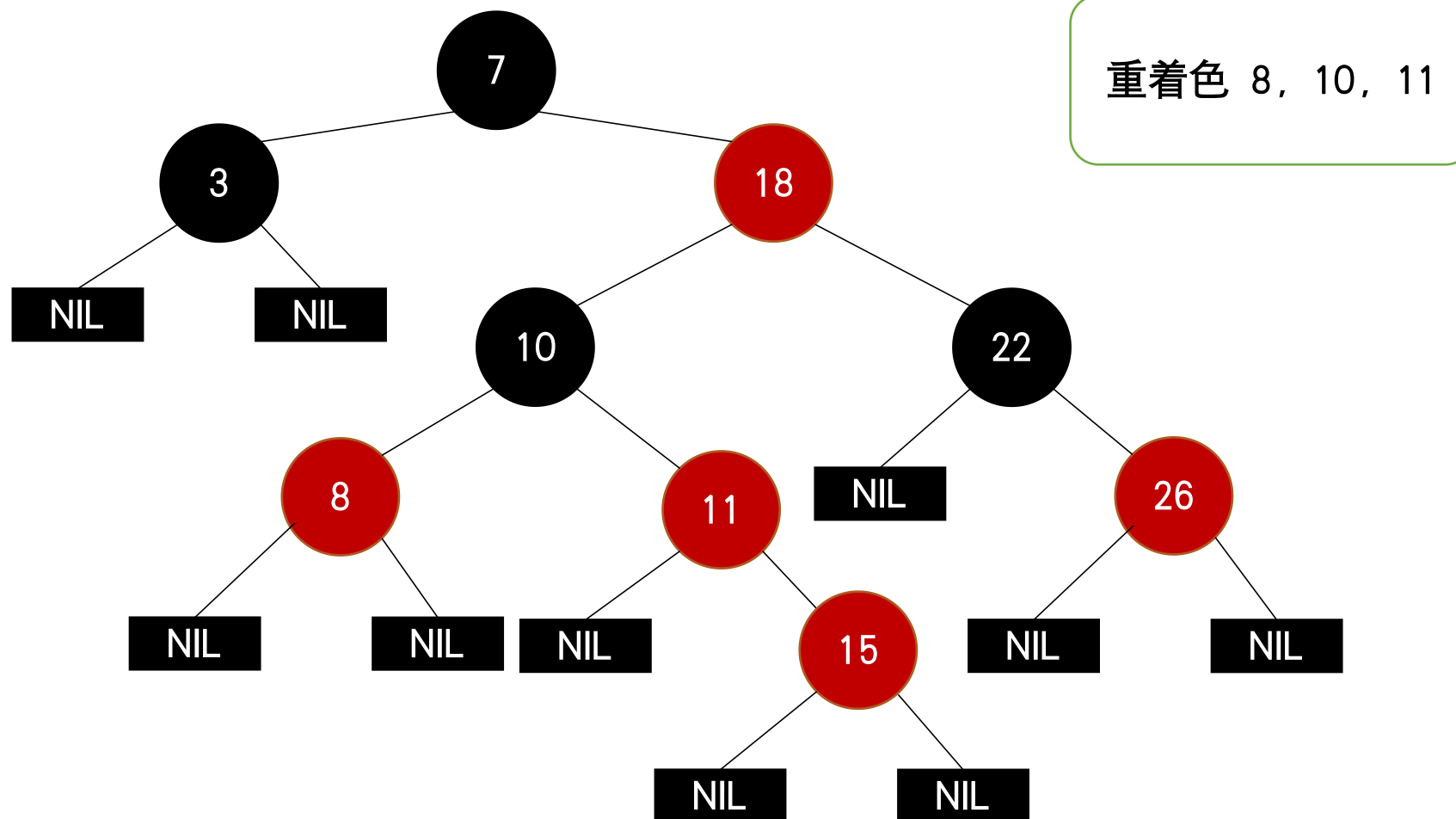
- 像BST一样搜索，查询成本为 $O(\log n)$



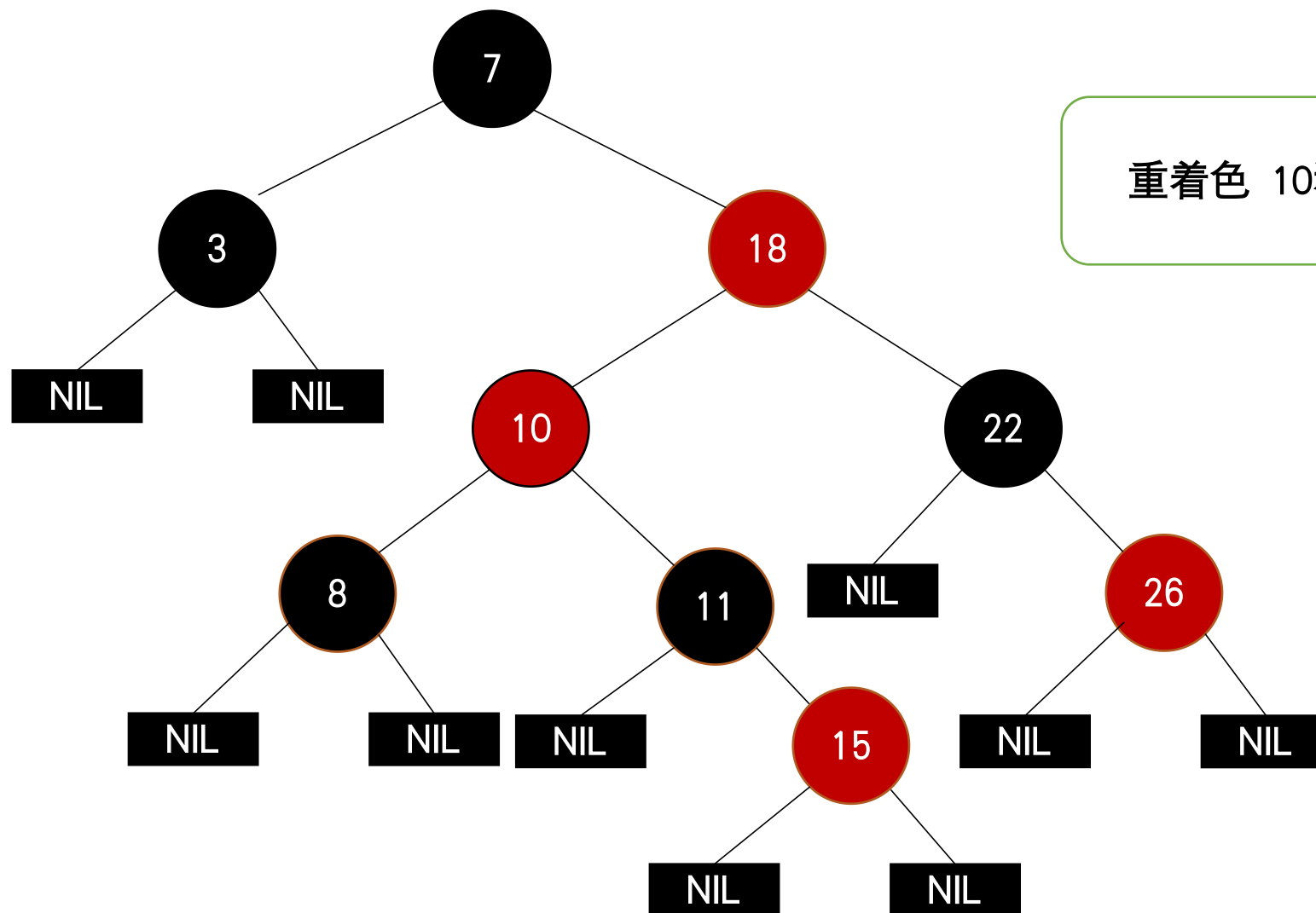
示例插入15



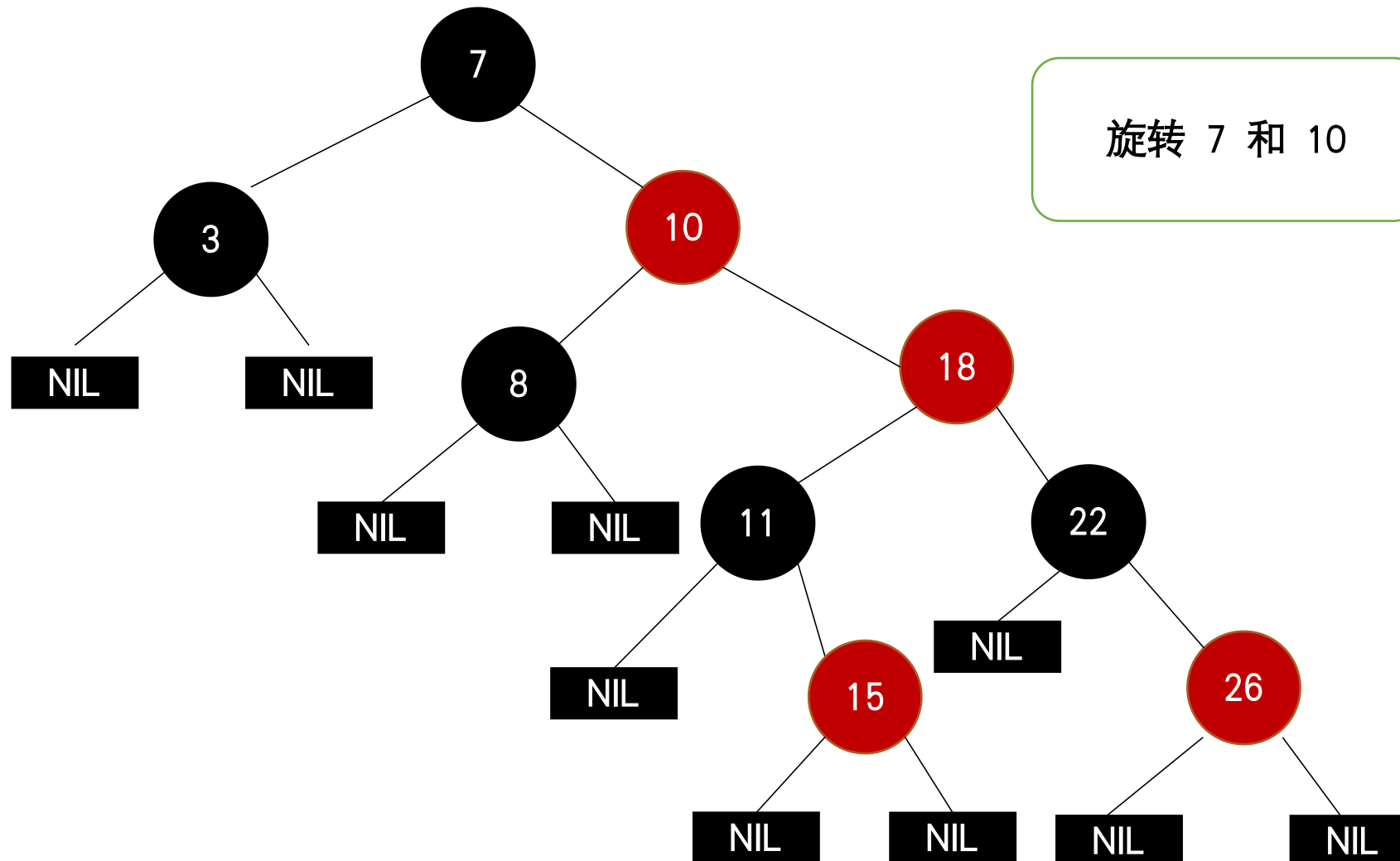
- 像BST插入一样插入，然后标记为红色



示例插入15



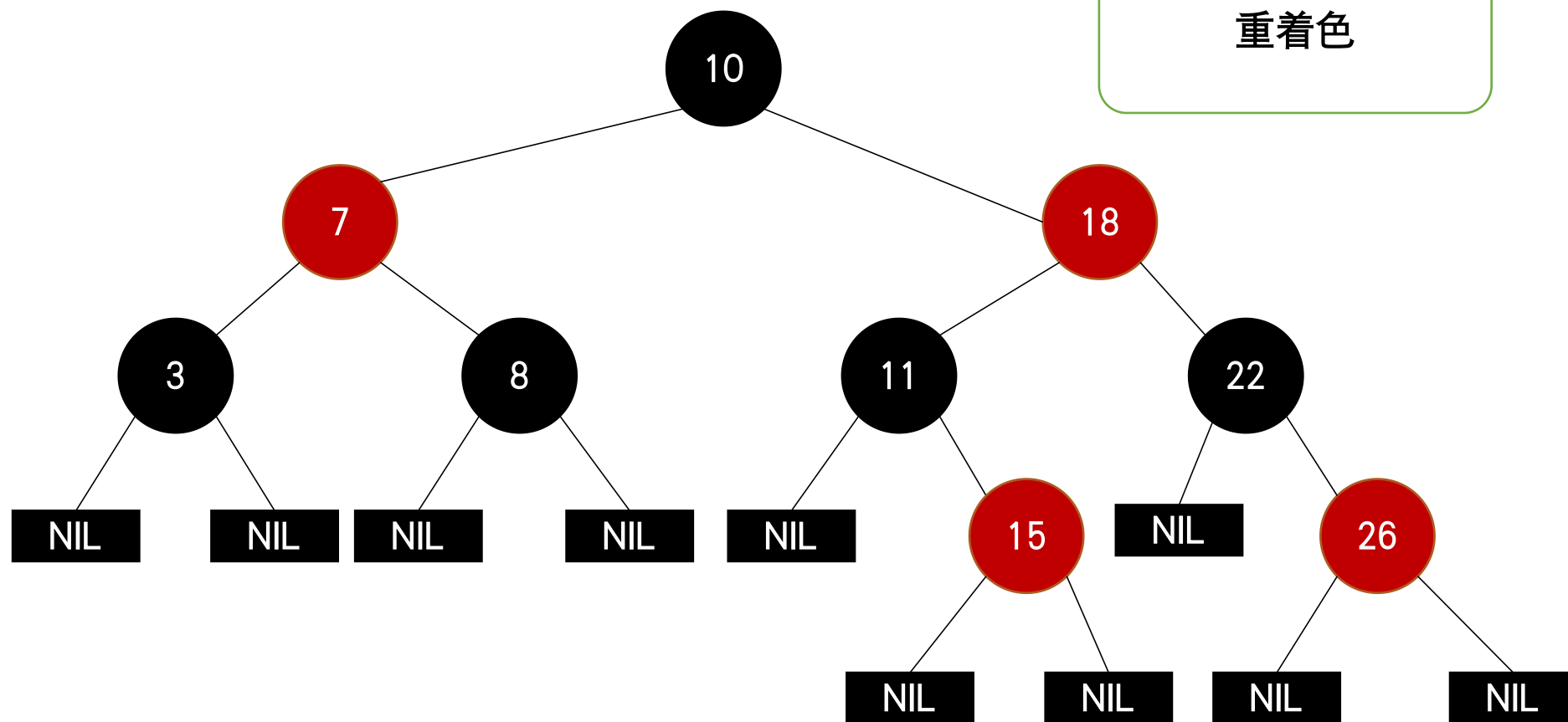
示例插入15



示例插入15



重着色



红黑树插入



RB-INSERT(T, z)

```
1  y = T.nil
2  x = T.root
3  while x ≠ T.nil
4      y = x
5      if z.key < x.key
6          x = x.left
7      else x = x.right
8  z.p = y
9  if y == T.nil
10     T.root = z
11 elseif z.key < y.key
12     y.left = z
13 else y.right = z
14 z.left = T.nil
15 z.right = T.nil
16 z.color = RED
17 RB-INSERT-FIXUP(T, z)
```

• 更新成本为
 $O(\log n)$

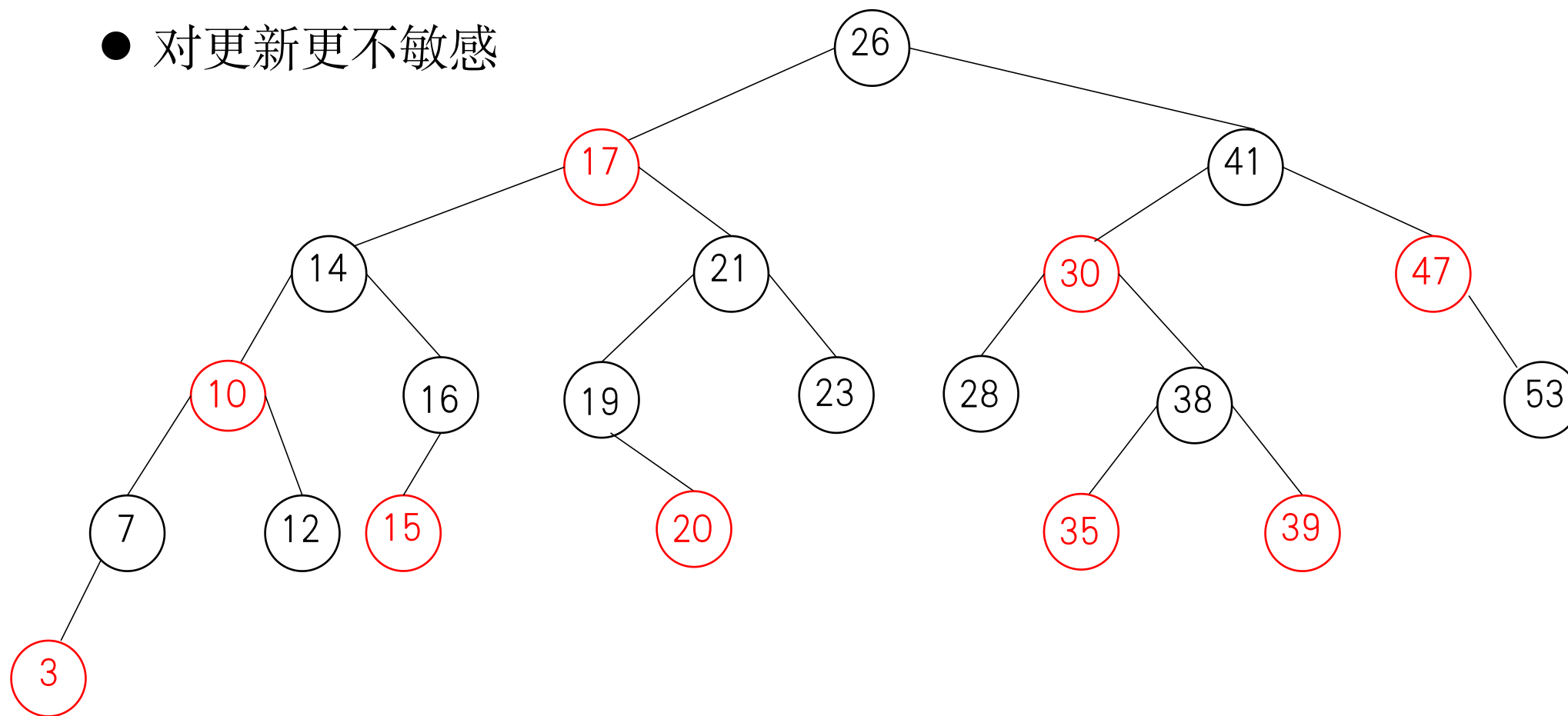
RB-INSERT-FIXUP(T, z)

```
1  while z.p.color == RED
2      if z.p == z.p.p.left
3          y == z.p.p.right
4          if y.color == RED
5              z.p.color = BLACK      // case1
6              y.color = BLACK        // case1
7              z.p.p.color = RED      // case1
8              z = z.p.p              // case1
9      else if z == z.p.p.right
10         z = z.p                    // case2
11         LEFT-ROTATE(T, z)         // case2
12         z.p.color = BLACK          // case3
13         z.p.p.color = RED          // case3
14         RIGHT-ROTATE(T, z.p.p)    // case3
15     else (same as then clause with “right” and “left”
        exchanged)
16  T.root.color = BLACK
```

红黑树对比AVL树



- 对更新更不敏感



二叉搜索树的优化



- **好消息：** 若随机插入元素，BST 性能优异，每次操作的时间复杂度为 $\Theta(\log N)$ 。
- **坏消息：** 实际应用中，我们无法总以随机顺序插入。
- 比如，数据可能是随时间陆续到达的，无法一次性获取全部数据，示例：存储事件时间戳

`add("01-Oct-2026, 10:31:00")`

`add("01-Oct-2026, 18:51:00")`

`add("02-Oct-2026, 00:05:00")`

`add("02-Oct-2026, 23:10:00")`

二叉搜索树的优化——如何避免失衡?



问题根源:

二叉搜索树失衡的根源在于：总是在底部添加新叶子节点（导致树不断长高、倾斜）。

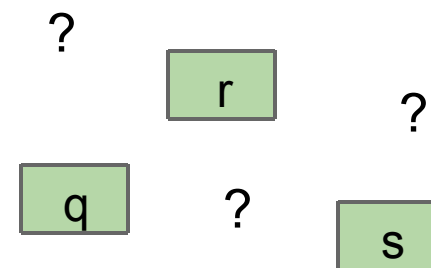
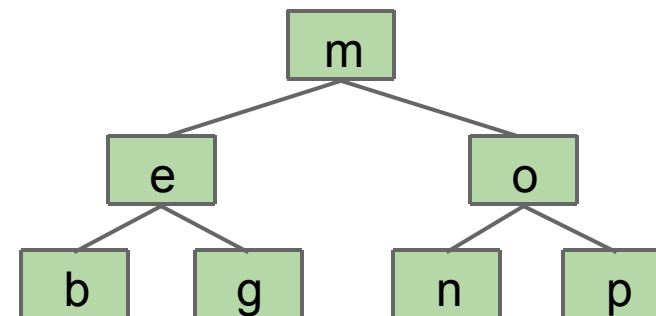
优化思路:

永远不要在底部添加新叶子节点!

这样树就永远不会失衡。

新的挑战:

那么，对于源源不断到来的新键，我们该如何处理？



二叉搜索树的优化——通过“超载”避免失衡



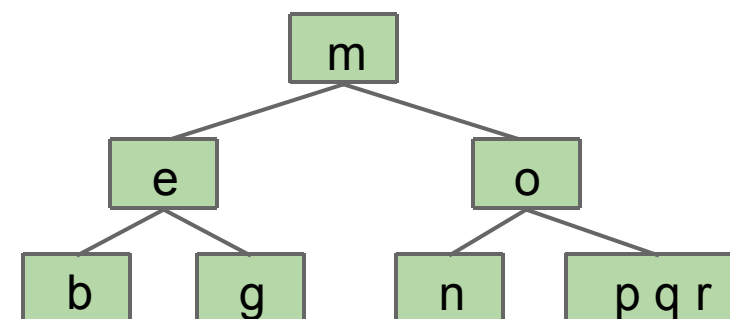
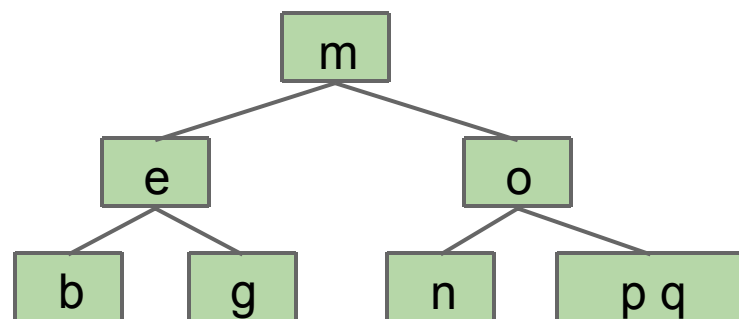
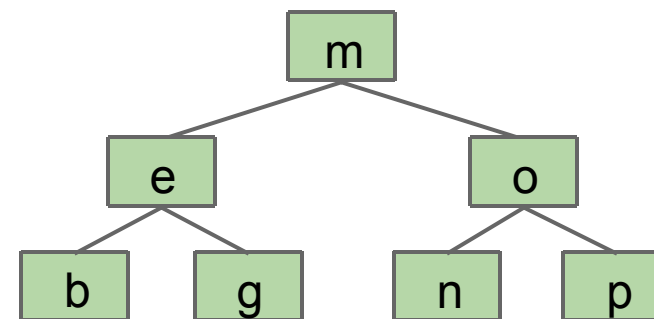
问题根源:

BST 失衡的根源在于：总是在底部添加新叶子节点。

优化策略:

通过“超载”叶子节点来避免产生新叶子；“超载树”始终保持平衡，因为叶子节点的深度不会改变。

树的高度 = 最大深度 (max depth) 。

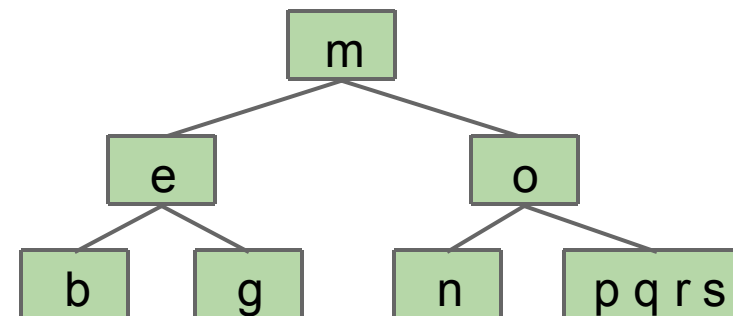


二叉搜索树的优化——通过“超载”避免失衡



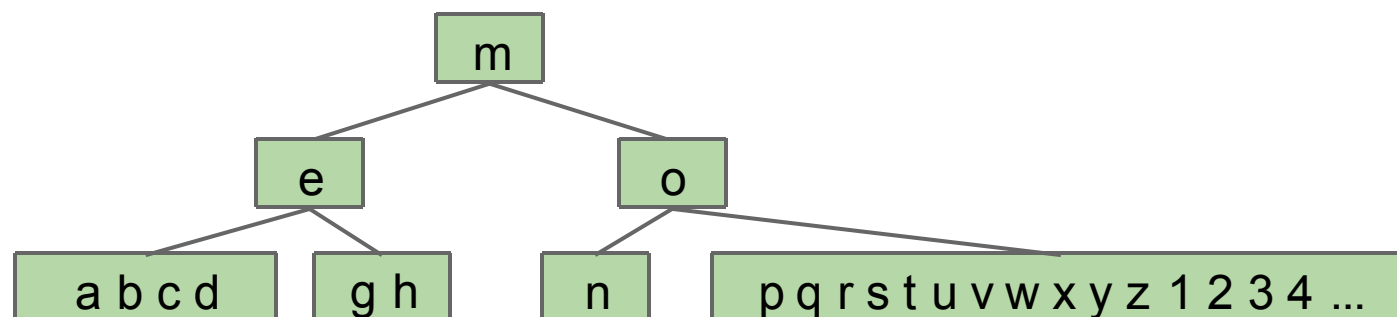
特点： 超载树在逻辑上是自洽的，但结构非常怪异。

查找示例： 查找r



问题： 这个方案有什么缺陷？

超载会导致叶子节点无限膨胀，查找效率退化成链表 $O(N)$

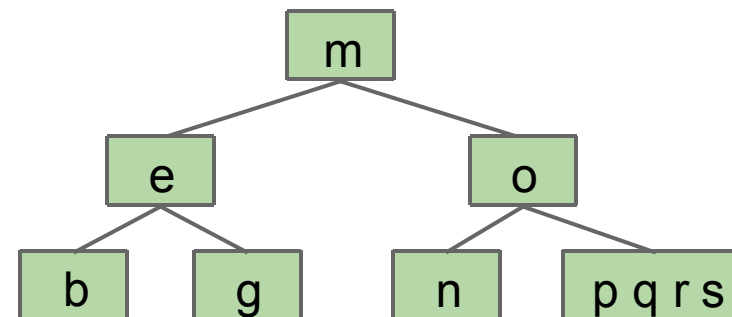


二叉搜索树的优化——超载树的困境与应对

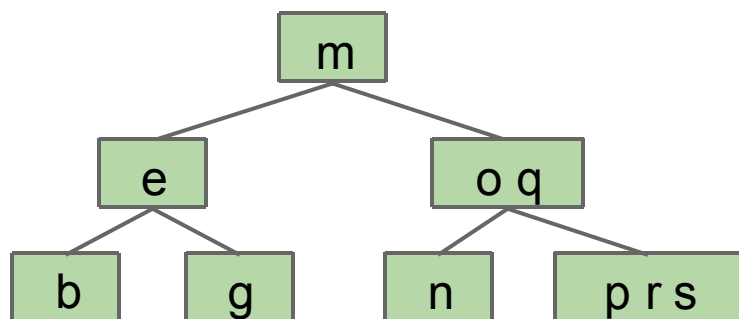


新问题：叶子节点会变得过于臃肿。

解决方案：设定节点容量上限（如 3 个元素）；溢出时，将中间元素上交给父节点，即选择“左中间”元素上移



新问题：o 和 q 之间的元素放哪？

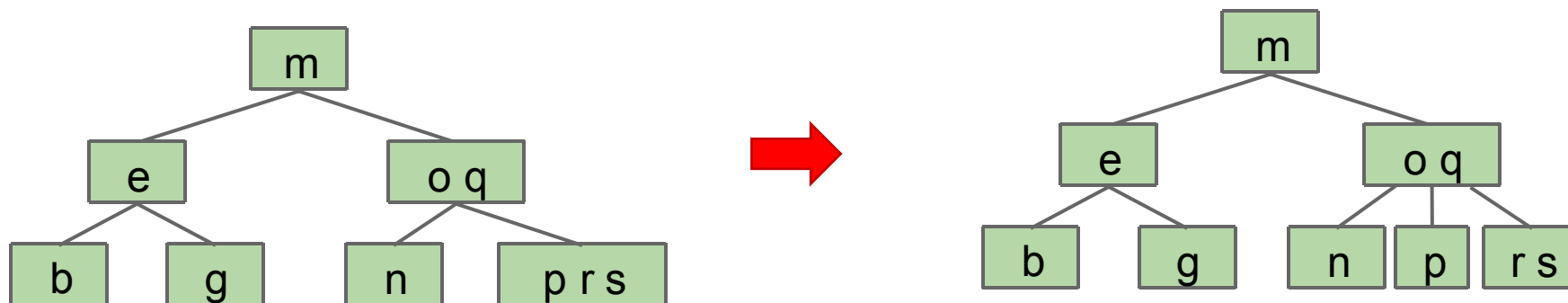


二叉搜索树的优化——超载树的困境与应对



新问题： 溢出时元素放置， o 和 q 之间的元素放哪？

解决方案： 溢出时，抽取中间元素上交给父节点，该节点随之分裂为左右两半，父节点由此拥有三个子节点



二叉搜索树的优化——逻辑自洽的“超载树”



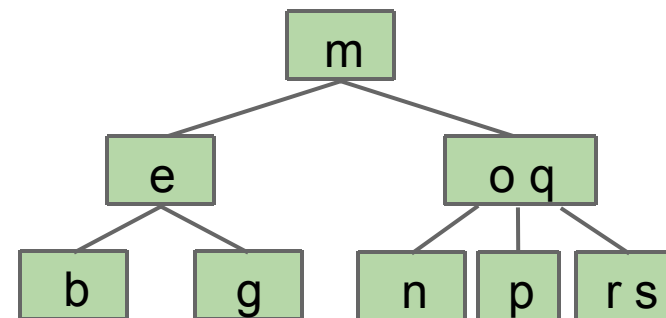
这是一种逻辑自洽，且不那么奇怪的数据结构。

查找示例： contains(r)

$r > m \rightarrow$ 向右走

$r > o \rightarrow$ 与 q 比较

$r > q \rightarrow$ 向右走



性能说明： 检查一个节点内部最多需要 Z 次比较，但由于 Z 有上限，所以这是可以接受的。

(Z 代表节点中的元素数量)

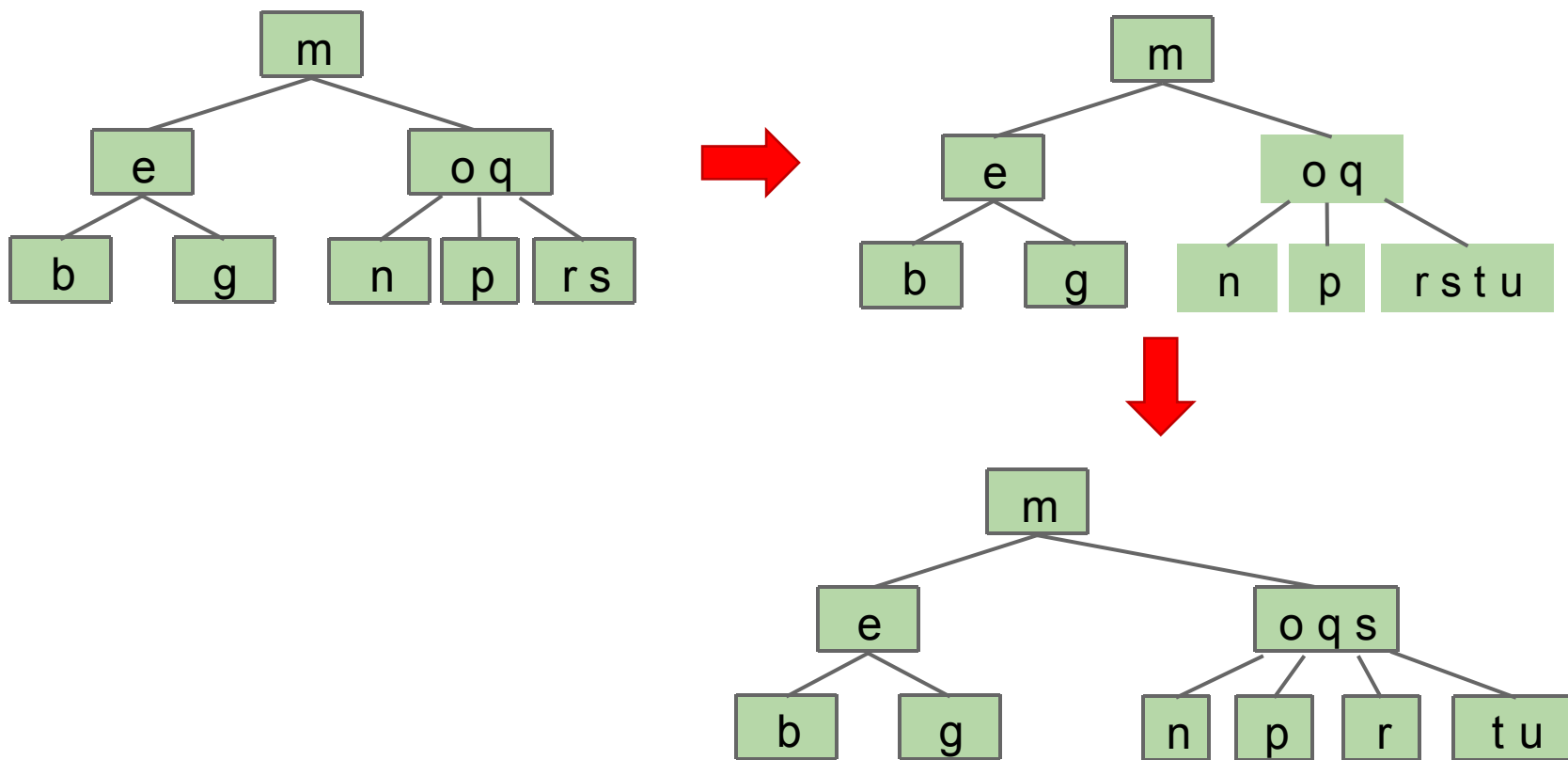
进阶问题：

如果一个非叶节点也过度膨胀，我们也能分裂它吗？

二叉搜索树的优化——非叶子节点超载问题



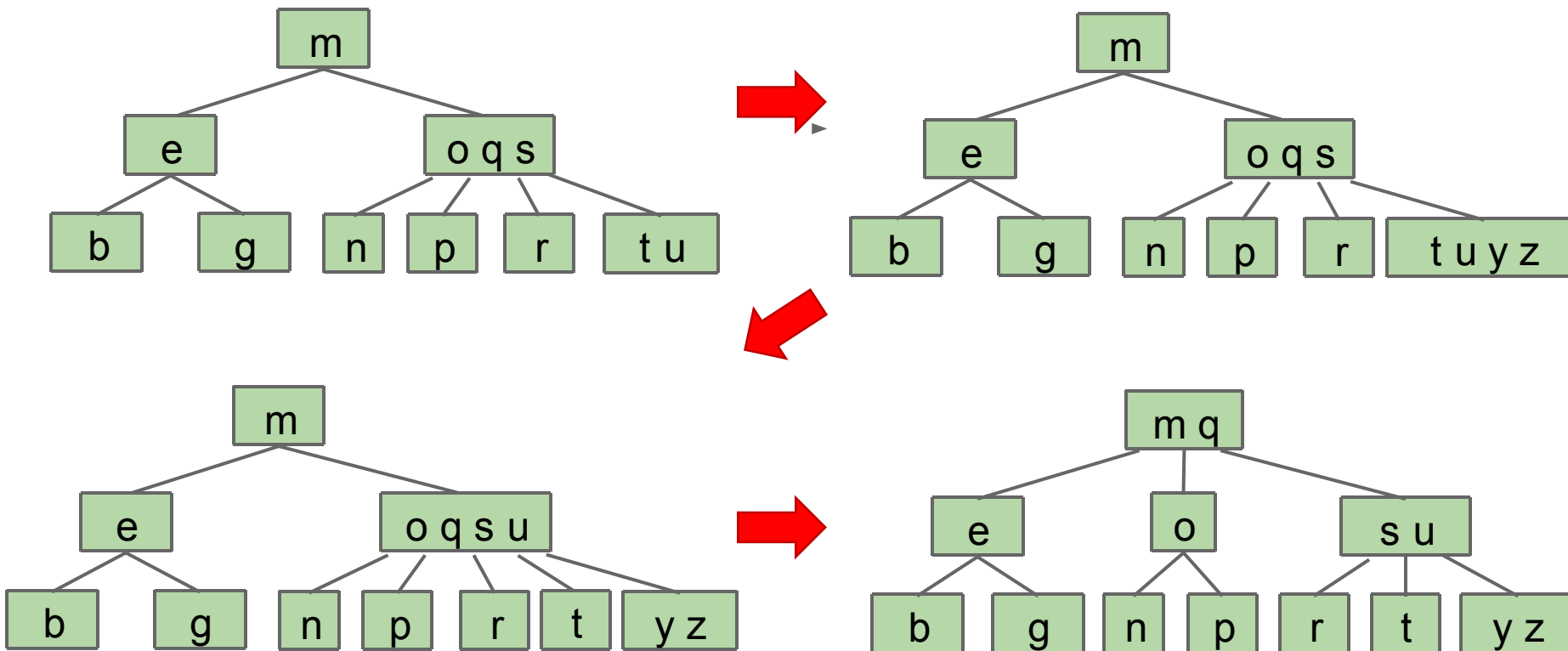
假设继续插入t和u



二叉搜索树的优化——非叶子节点超载问题



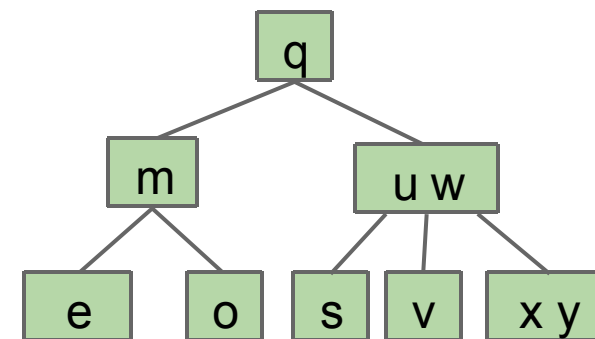
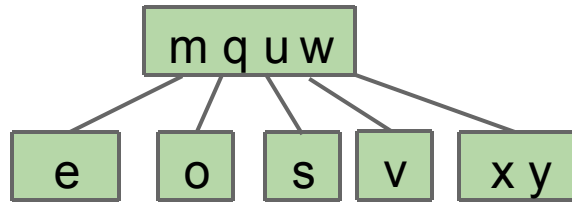
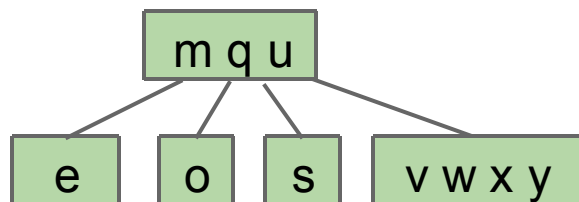
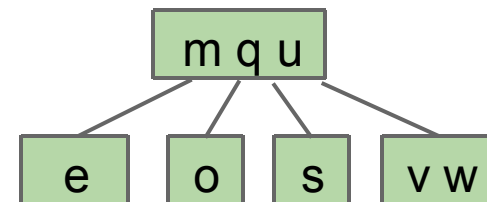
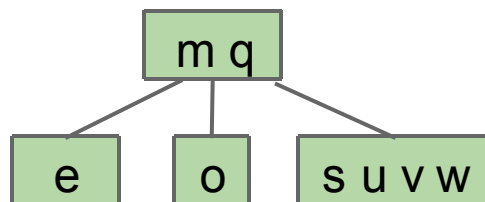
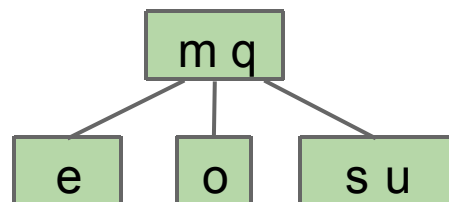
假设继续插入y和z



二叉搜索树的优化——根节点超载问题



如果根节点（Root）也装满并溢出了，该如何处理？



二叉搜索树的优化——分裂树的平衡性



关键观察:

分裂树天然保持完美平衡。

分裂根节点: 所有节点整体下移一层

分裂叶子/内部节点: 树高不变

核心结论:

所有操作的时间复杂度均保证为 $\Theta(\log N)$ 。

二叉搜索树的优化——关键参数与性能总结



核心参数:

M = 最大子节点数 (元素上限 + 1)

树高 = 介于 $\log_M(N)$ 与 $\log_2(N)$ 之间

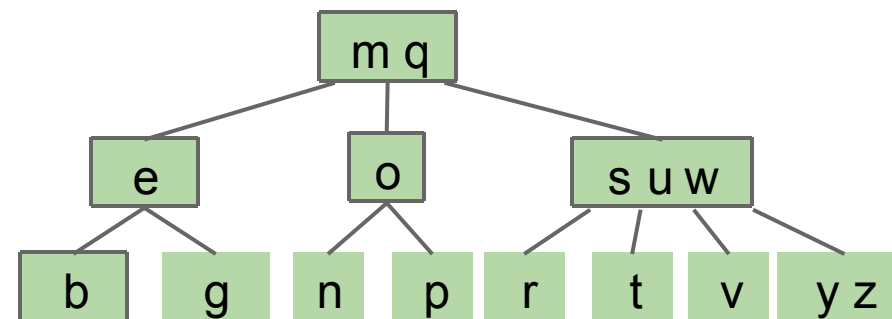
操作成本:

单次插入最多分裂 H 次

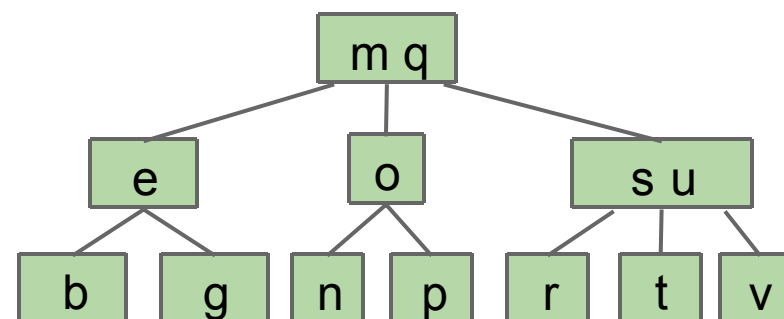
时间复杂度:

插入 / 查找 = $\Theta(MH) = \Theta(M \log N)$

因为 M 是常量, 所以 $\Theta(\log N)$



当 $M = 4$ 时:
每个节点最多容纳 3 个元素。
每个节点最多拥有 4 个非空子节点。



当 $M = 3$ 时:
每个节点最多容纳 2 个元素。
每个节点最多拥有 3 个非空子节点。

B树



上面的树结构正式名称——**B树**。

- 阶为 $M=4$ 的B树也被称为 2-3-4树 或 2-4树，这个命名指的是节点可以拥有的子节点数量。例如，2-3-4树的节点可以有2、3或4个子节点。
- 阶为 $M=3$ 的B树也被称为 2-3树。

关于“B树”中“B”的起源，其发明者从未给出过解释。正如我们将看到的，“*balanced*（平衡）”、“*broad*（宽）”、“*bushy*（茂密）”似乎都说得通。也有人认为“B”代表波音公司（*Boeing*）。不过，鉴于 *Bayer* 的贡献，将B树理解为“*Bayer*树”似乎也颇为恰当。

——*Douglas Corner*（《无处不在的B树》）

B树



B树的两种典型应用场景:

- 小 L ($L=2$ 或 $L=3$)

用作概念上简单的平衡搜索树

- L 非常大 (如数千)

实际应用于数据库和文件系统 (即存储大记录的系统)

B树



定义： 允许每个节点包含最多 $M - 1$ 个键-链接对，从而将2-3树推广为 B树。

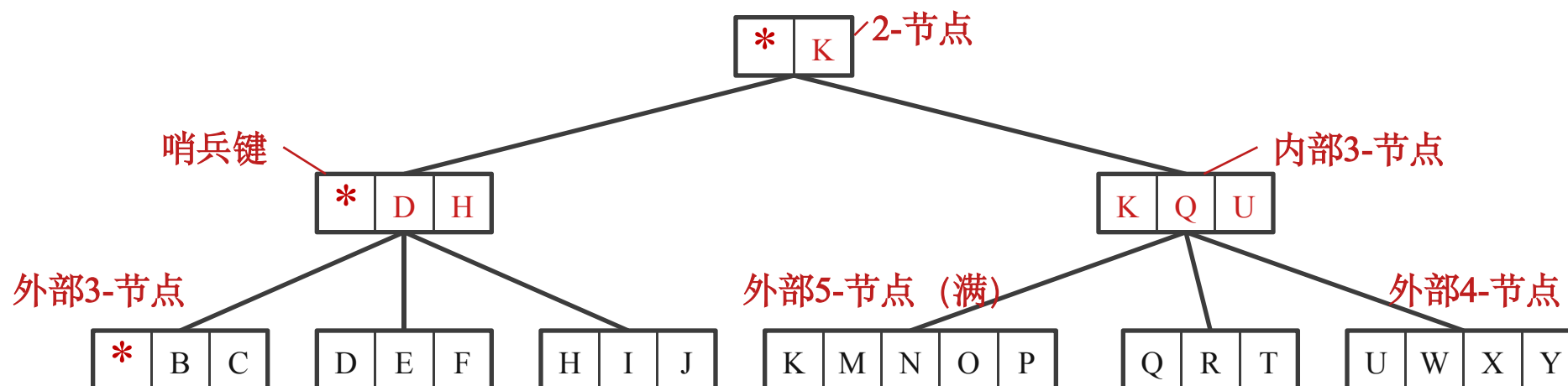
结构约束：

- 根节点：至少 2 个键-链接对
- 其他节点：至少 $M/2$ 个键-链接对
- 外部节点：包含客户端键
- 内部节点：包含用于引导搜索的键副本

参数选择：

M 尽可能大，使 M 个链接恰好装入一个页面（如 $M = 1024$ ）。

B树的结构剖析 ($M = 6$)



- 每个红色键是子树中最小键的副本
- 除根节点外，所有节点都是3-、4-或5-节点
- 客户端键（黑色）位于外部节点中

文件系统模型——为什么B树要这样设计？



文件系统基本概念：

页 (Page)：连续的数据块（如 4,096 字节）

探测 (Probe)：对页的首次访问（如从磁盘读取到内存）

核心性质：一次探测的时间 **远大于** 访问页内数据的时间（磁盘慢，内存快）。

代价模型：探测次数 (Number of probes)

目标：用最少的探测次数完成数据访问。

文件系统模型——为什么B树要这样设计?



文件系统与B树的联系:

回顾B树定义: 每个节点可容纳最多 $M - 1$ 个键, M 个链接能装入一个页 (如 $M = 1024$)

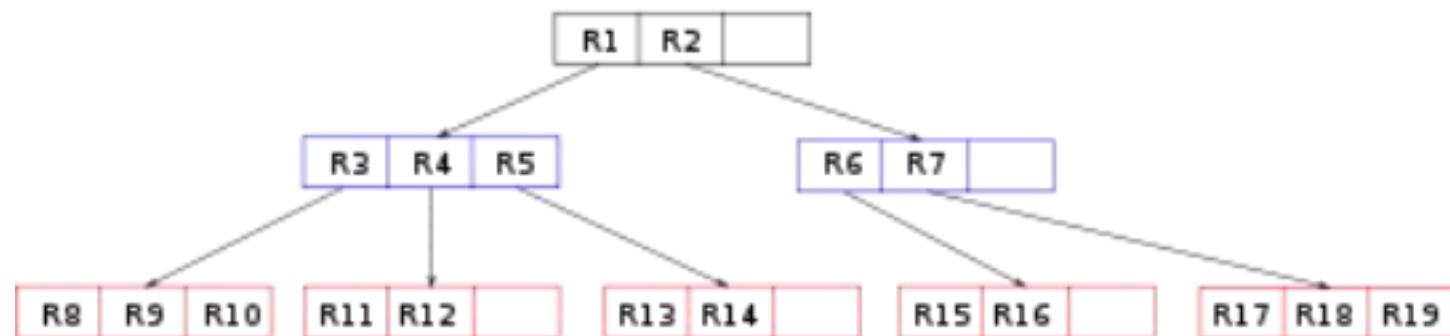
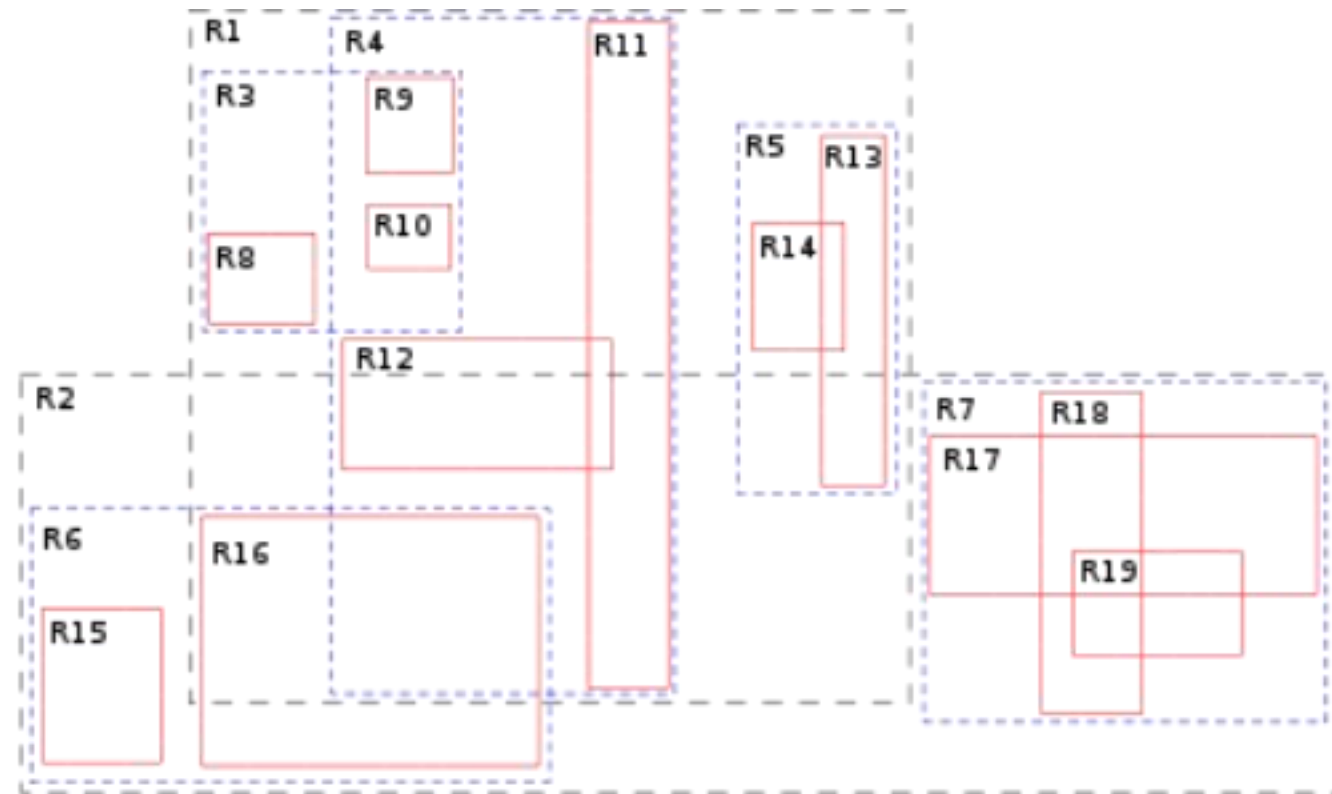
设计动机: 一次探测 = 读取一个节点 $\rightarrow$ 让每个节点尽可能大, 就能在一次磁盘I/O中读取更多键

效果: 树高大幅降低 $\rightarrow$ 探测次数 = 树高 = $\Theta(\log_M N)$, 远优于二叉搜索树的 $\Theta(\log_2 N)$



- R树是一种用于空间访问方法的树形数据结构，即用于索引多维信息，如地理区域。
- 那么空间坐标数据访问有什么需求呢？

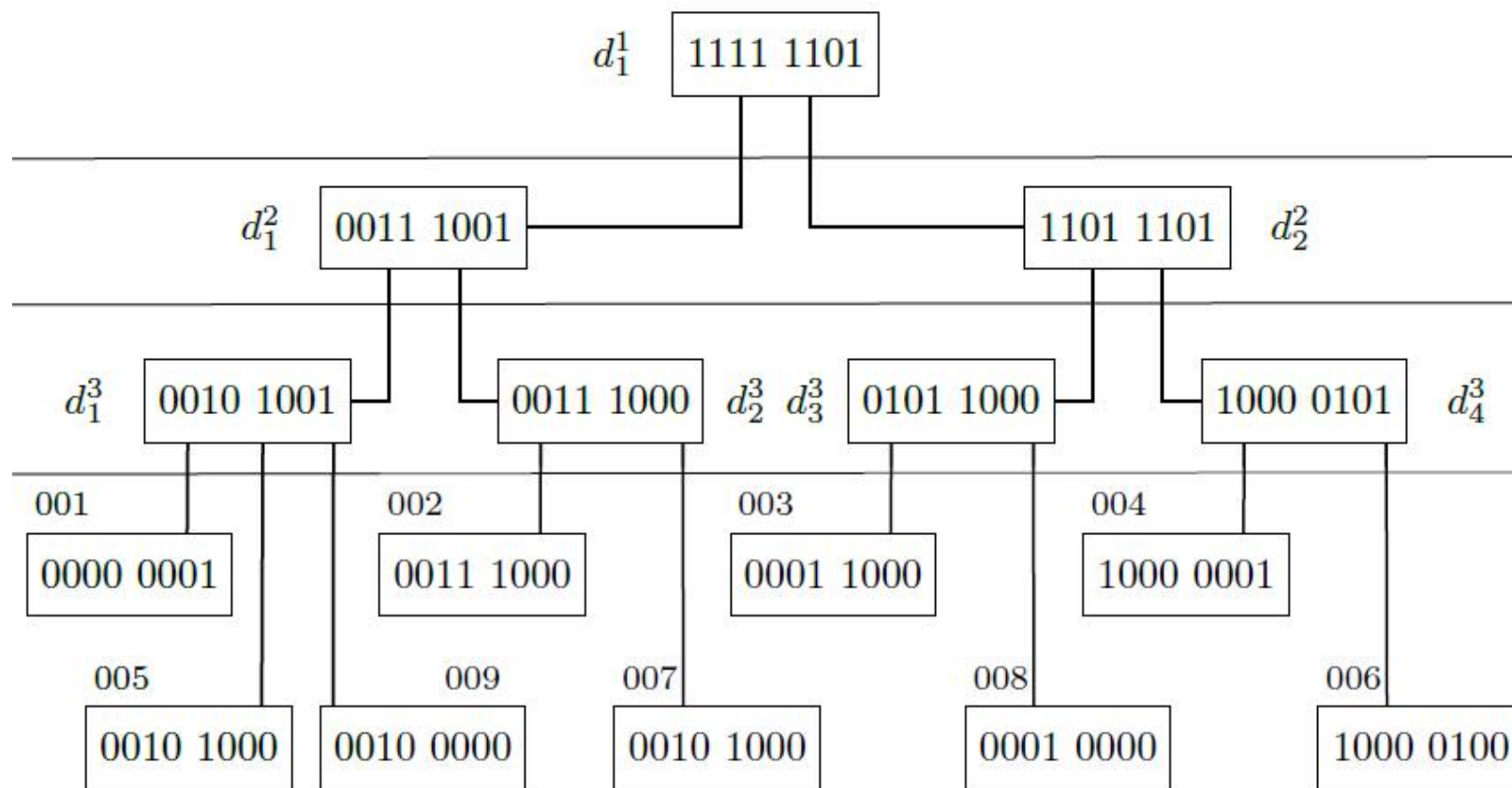




签名树



- 由相应的叶节点指向的签名。





湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——